



Rapport de stage

CATIA Electrical Logical System Design:

Automatisation d'allocation de câbles aux connecteurs
de coupure

Elève : HUANG He
Tuteur : Sébastien GOUTAUDIER

Table des matières

Remerciements.....	3
Introduction	4
Présentation de l'entreprise : Dassault Systèmes.....	5
Une présence mondiale	6
Une vision forte : le virtuel au service du réel	6
Une culture d'innovation portée par l'esprit « IFWE »	7
Une stratégie autour de l'humain, l'industrie et les expériences.....	7
L'Equipe CATIA – Schématique Electrique	7
Mission du stage	11
Contexte.....	11
Net to Wire	11
Connecteur de coupure et branchement des câbles.....	13
Objectif du stage.....	14
Phase 1 : Conception d'algorithme	15
Modélisation de la problématique	15
La notion de proximité	16
Le croisement des câbles.....	18
Stratégie de recherche	20
Phase 2 : Implémentation d'un prototype	23
Auto-détections des cavités	23
Conception de la structure de données.....	24
Triangulation Delaunay	25
Recherche DFS	25
L'évaluation d'une solution	26
Visualisation.....	26
Phase 3 : Implémentation d'une commande CATIA	27
Introduction d'infrastructure	27
Solution existante	28
Création d'une nouvelle commande CATIA.....	29
L'interface utilisateur et l'introspection.....	30
Nouvelle classe de cavité.....	32
Bilan technique et professionnel.....	34
Conclusion	36
Annexes	37
Annexes A : Référence.....	37
Annexes B : Code d'algorithmes	38
Triangulation Delaunay : Algorithme Boyer - Watson.....	38
Smallest enclosing circle : Algorithme Welzl	38
Annexes C : Extension d'infrastructure DS	39
std::erase_if.....	39
std::vector<T,Allocator>::emplace_back	40
Comparaison de nombre à virgule flottant et opérateur « <=> »	40

Remerciements

Je tiens à remercier chaleureusement Monsieur Sébastien GOUTAUDIER, mon manager et mon tuteur, pour son accompagnement tout au long de mon stage. Son aide et conseil, son écoute et sa confiance ont grandement facilité mes activités au sein de Dassault Systèmes et m'ont permis de monter en compétences rapidement afin de mener au mieux ma mission.

Je remercie également très sincèrement Monsieur Mathieu LAPOINTE, pour la formation complète et structurée qu'il m'a dispensée sur le langage de programmation C/C++, ainsi que sur les infrastructures de développement au sein de Dassault Systèmes. Son expertise et sa pédagogie m'ont permis de mieux comprendre les fondements techniques de l'environnement dans lequel j'évoluais et les outils nécessaires pour ma mission, ce qui a largement contribué à la qualité de mes réalisations.

Je souhaite également exprimer ma reconnaissance à Monsieur Maxime MOSTKOWY, pour le temps qu'il m'a accordé dans la relecture et l'amélioration de ce rapport, ainsi que pour les conseils précieux qu'il m'a donnés concernant mon avenir professionnel.

Enfin, je souhaite remercier l'ensemble de mes collègues dans le bureau de Dassault Systèmes Lyon qui m'ont aidé pendant mon stage, pour leur accueil chaleureux, leur disponibilité et les nombreux échanges constructifs que nous avons partagés. Leur esprit d'équipe et leur expertise m'ont offert un environnement de travail très agréable sur le plan humain et professionnel.

Introduction

Ce stage de fin d'études s'est déroulé au sein de Dassault Systèmes, le leader mondial dans le domaine des logiciels de conception 3D, de simulation et de gestion du cycle de vie des produits (PLM).

Intégré à l'équipe de CATIA Schématique Electrique, j'ai contribué à l'automatisation et à l'amélioration d'un module de routage de câbles dans les interfaces de coupures, dans un contexte de maquette numérique appliquée à l'aéronautique et à l'automobile.

L'objectif principal de la mission est de concevoir, expérimenter et valider un algorithme capable de connecter automatiquement des câbles à une série de connecteurs de coupure en respectant certaines contraintes techniques.

Pour atteindre cet objectif, des algorithmes ont été adaptés à la problématique. Une phase de validation par un prototype a ensuite permis de tester la robustesse, la précision et la pertinence de la solution développée. Finalement, cette fonctionnalité a été mise en œuvre au sein de la plateforme 3DEXPERIENCE CATIA.

Ce stage m'a offert l'opportunité d'approfondir mes compétences en algorithmique appliquée à la CAO, en optimisation géométrique, ainsi qu'en ingénierie logicielle dans un cadre industriel exigeant.

Une piste explorée au cours de la mission a été l'utilisation de techniques d'intelligence artificielle pour résoudre le problème de routage. Cependant, cette approche s'est heurtée à un manque de données d'apprentissage et à l'impossibilité de démontrer sa fiabilité dans un contexte industriel exigeant. La bonne nouvelle est que l'algorithme classique développé dans ce projet fonctionne correctement, même s'il reste encore des axes d'amélioration possibles.

Il a eu pour moi une résonance toute particulière : presque tous les étudiants de l'UTT apprennent à utiliser CATIA durant leur cursus. J'avais moi-même manipulé ce logiciel en tant qu'étudiant. Pour moi, contribuer aujourd'hui à son développement est à la fois inattendu, intéressant et très formateur.

Mots-clés :

- CATIA Electrical System Design
- Graph de proximité
- Optimisation combinatoire
- C/C++

Présentation de l'entreprise : Dassault Systèmes

Dassault Systèmes est un leader mondial des solutions numériques innovantes, spécialisé dans la création de mondes virtuels au service du monde réel. Depuis sa création en 1981, Dassault Systèmes a évolué d'une modeste équipe d'une vingtaine d'ingénieurs spécialisés dans l'industrie aéronautique à un acteur mondial incontournable de la transformation numérique. Voici une petite histoire de son parcours :

CATIA - 3D Design (1981) : Dassault Systèmes a été fondé à la suite de l'essaimage d'une équipe d'ingénieurs de Dassault Aviation, dans le but de développer des solutions de conception en trois dimensions. Cette initiative a donné naissance à CATIA (Conception Assistée Tridimensionnelle Interactive Appliquée), la solution emblématique de l'entreprise, dédiée à la Conception Assistée par Ordinateur (CAO), notamment pour les industries aéronautique et automobile.

ENOVIA - Solution PLM (1999) : Acquisition de la solution Product Manager Software d'IBM, qui deviendra ensuite ENOVIA, afin de gérer les données produites par CATIA pour les clients majeurs et d'étendre les capacités du groupe en termes de gestion du cycle de vie des produits (PLM), transformant ainsi les opportunités de marché existantes en avantages concurrentiels.

3DEXPERIENCE (2012) : La plateforme 3DEXPERIENCE voit le jour pour aider les entreprises à repenser leurs opérations à l'aide d'une plateforme qui connecte contributeurs, idées, données et solutions au sein d'un seul et même environnement collaboratif. Lancement des premières Industry Solution Experiences (ISE) – des collections prédéfinies d'applications Dassault Systèmes qui répondent aux besoins spécifiques des secteurs industriels et offrent une valeur ciblée aux clients et/ou à leurs utilisateurs finaux.

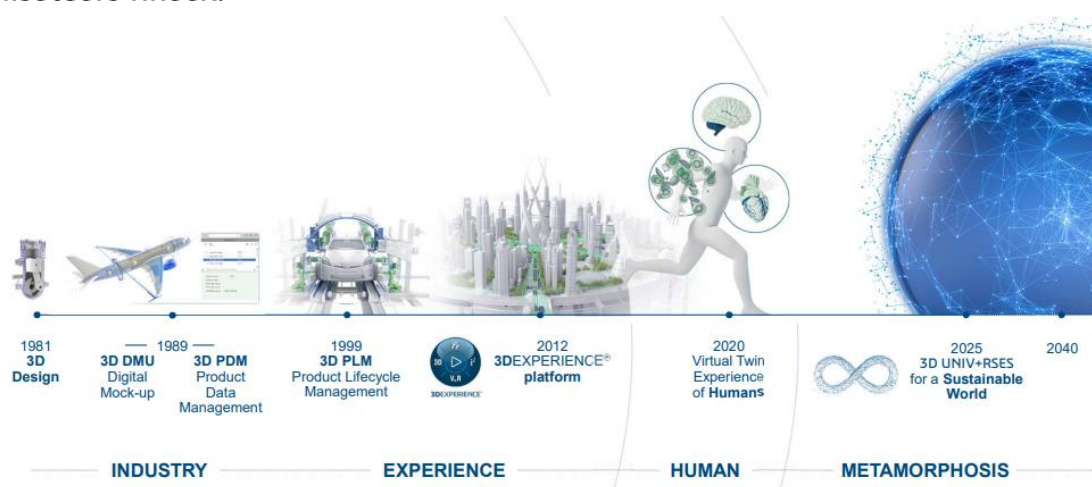


Figure 1 : Histoire de Dassault Systèmes

Une présence mondiale

Dassault Systèmes emploie environ 25 000 collaborateurs passionnés, répartis sur les cinq continents, avec une forte concentration en Europe (41 %), en Asie-Pacifique (32 %) et en Amériques (27 %). Ces collaborateurs partagent une vision commune : améliorer la vie réelle grâce à l'innovation dans les mondes virtuels. Cette force de travail internationale est soutenue par un réseau de 77 laboratoires de recherche et développement, ainsi que par un vaste écosystème de plus de 14 000 partenaires technologiques, commerciaux et éducatifs, et plus de 200 partenaires scientifiques et de recherche. Ces collaborations renforcent les capacités d'innovation collective et élargissent la portée mondiale de l'entreprise.

Le capital de Dassault Systèmes est majoritairement détenu par le Groupe Industriel Marcel Dassault, contrôlé par la famille Dassault, ainsi que par la direction de la société. L'entreprise dispose de sièges régionaux stratégiques à Boston, Paris et Shanghai, assurant un support local et une proximité avec ses clients dans le monde entier.

Une vision forte : le virtuel au service du réel

Plus qu'un simple éditeur de logiciels, Dassault Systèmes est un partenaire stratégique qui conjugue science, technologie et innovation pour enrichir l'expérience humaine. À travers un portefeuille diversifié de 13 marques expertes, couvrant la conception 3D, la maquette numérique (DMU), la gestion du cycle de vie des produits (PLM) et bien d'autres domaines, l'entreprise accompagne des acteurs industriels majeurs dans la définition de nouvelles stratégies. Elle les guide aussi dans la mise en œuvre d'approches plus durables, vertes et écoresponsables, en phase avec les enjeux environnementaux actuels.

Dassault Systèmes est profondément convaincue que les mondes virtuels permettent de prolonger et d'améliorer le monde réel. Grâce à ses univers 3DEXPERIENCE, elle offre aux entreprises et aux individus des environnements collaboratifs pour imaginer des innovations durables, capables d'harmoniser produit, nature et vie. La plateforme 3DEXPERIENCE®, cœur de son offre, permet la conception de jumeaux numériques (ou jumeaux virtuels) d'un produit, d'un processus industriel, voire d'un système complet. Ces jumeaux numériques reproduisent fidèlement les caractéristiques physiques et fonctionnelles, offrant un espace d'expérimentation sécurisé, économe en ressources et respectueux de l'environnement.

Cette approche innovante s'inscrit dans une démarche de développement durable où l'entreprise mesure l'empreinte écologique et l'impact positif de ses solutions, ce que Dassault Systèmes appelle « l'éco-bilan ». Cette méthode permet d'aider les clients à transformer leur modèle économique vers une meilleure durabilité.

Une culture d'innovation portée par l'esprit « IFWE »

L'esprit « IFWE » est un concept propre à Dassault Systèmes :

- "IF WE ask the right questions, we can change the world."
→ Souligner l'importance de la curiosité et de la réflexion stratégique.
- "IF WE design without limits, can we create a better life?"
→ Encourager l'innovation dans le design au service de l'humain.
- "IF WE use nature as inspiration, can we build more sustainable cities?"
→ Allusion à l'approche bio-inspirée de la conception.
- "IF WE can dream it, we can make it."
→ Slogan plus global de l'entreprise, proche de sa philosophie.

Cet état d'esprit guide la culture interne et les ambitions externes de l'entreprise, résolument tournée vers l'innovation au service de l'humanité.

Une stratégie autour de l'humain, l'industrie et les expériences

Dassault Systèmes développe sa stratégie autour de trois axes :

- **Human** : l'humain est la ressource principale et la raison d'être de l'entreprise, avec une ambition d'apporter un bénéfice durable à la société par l'imagination, le savoir et le partage.
- **Industry** : offrir à chaque secteur industriel des solutions personnalisées pour atteindre des résultats durables et compétitifs.
- **Experiences** : promouvoir une nouvelle économie de l'expérience, où les solutions ne sont plus simplement des produits, mais des expériences à vivre.

L'Equipe CATIA – Schématique Electrique

J'ai effectué mon stage dans une équipe composée de sept personnes : un manager et six développeurs. Parmi ces développeurs, trois sont basés à Dassault Systèmes Lyon, tandis que les trois autres travaillent depuis la Belgique. Cette organisation répartie souligne l'importance du travail collaboratif à distance.

Nous travaillons principalement sur des machines à distance qui se situent dans le siège social de Dassault Systèmes à Vélizy-Villacoublay. Nous sommes sous la supervision de notre manager qui coordonne les projets et assure la cohérence des développements.

Une entreprise globale



25 000
collaborateurs



370 000+
clients, allant des entrepreneurs
aux multinationales, dans 12 industries



Répartition des effectifs :
41 % Europe
27 % Amériques
32 % Asie



184
sites



13
marques

Une entreprise innovante



+2 %
croissance des effectifs R&D

41 %
part des collaborateurs travaillant
dans les équipes R&D



839
innovations protégées



Au sein du **3DEXPERIENCE** Lab :
2 350
mentors impliqués

80
projets innovants et à fort impact
environnemental et sociétal
soutenus dans le monde depuis 2015

Une entreprise performante et en croissance

+5 %* **6,21 Mds €***
chiffre d'affaires total



+14 %*
croissance du chiffre d'affaires logiciel
3DEXPERIENCE



+10 %*
croissance du chiffre d'affaires logiciel
en souscription



+7 %*
croissance du chiffre d'affaires logiciel *cloud*



31,9 %*
de marge opérationnelle

+9 %*
hausse du bénéfice net dilué par action

* Non-IFRS, croissance à taux de change constants.
Voir les chapitres 1.7 et 3.1 pour les données IFRS.

Une entreprise durable et responsable



#4
dans le classement *S&P Global CSA*
du secteur des logiciels et aussi membre
du *Dow Jones Sustainability World Index*



AAA
classé "Leader" pour le secteur
des logiciels selon la notation MSCI



69,8 %
chiffre d'affaires éligible
à la Taxonomie européenne

35,0 %
chiffre d'affaires aligné
à la Taxonomie européenne



5
femmes parmi les 13 membres
de l'équipe de direction

Figure 2 : Chiffres Clés de Dassault Systèmes 2024

RESSOURCES ET CAPITAL

CAPITAL INTELLECTUEL

13 portefeuilles technologiques pour servir l'ensemble du cycle d'innovation
40+ années cumulées de savoir industriel
1 286 M€ d'investissements en R&D (+5 %)
839 innovations protégées

CAPITAL HUMAIN

25 000 collaborateurs
41 % en R&D
5 femmes parmi les 13 membres de l'équipe de direction
26 % de femmes au sein des *People managers*

CAPITAL SOCIAL

14 000+ personnes dans l'écosystème de partenaires commerciaux
200+ partenaires en recherche scientifique
10 000+ personnes dans l'écosystème de partenaires technologiques et *marketplace*

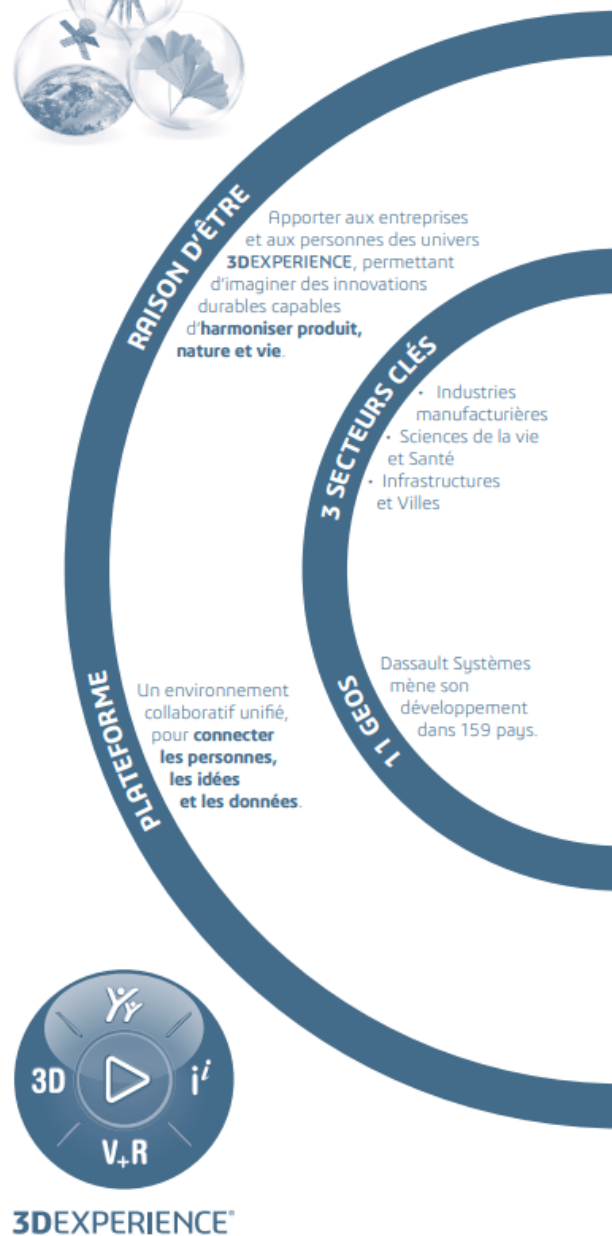
CAPITAL FINANCIER

Structure d'actionnariat stable et de long terme

1 459 M€ trésorerie nette
A Stable notation de crédit de S&P

CAPITAL NATUREL

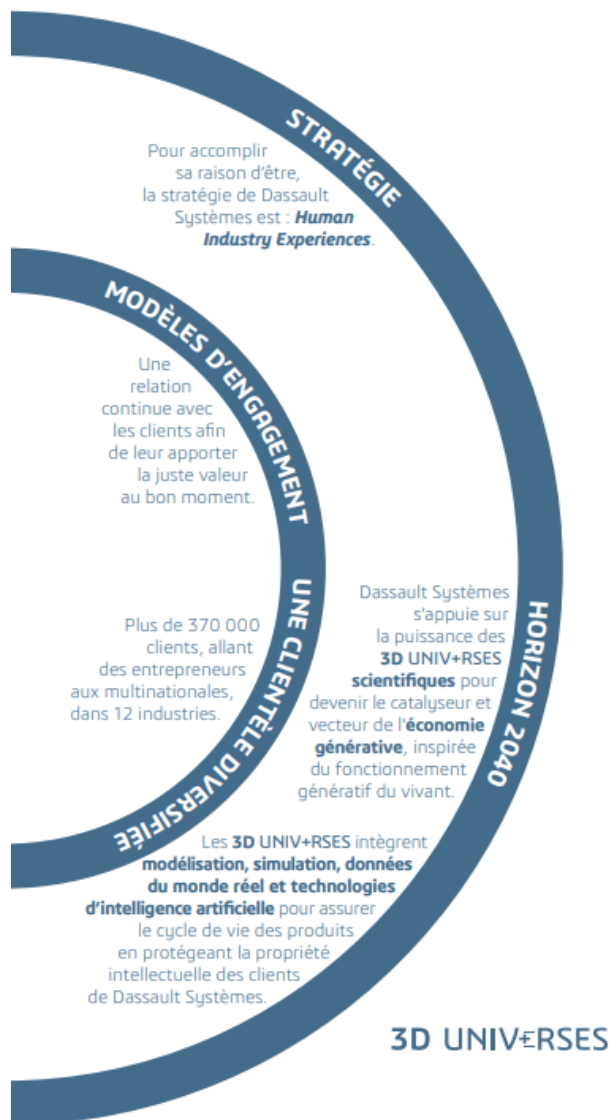
91 % part d'électricité renouvelable
47,9 % part des fournisseurs (en émissions CO₂) ayant défini des objectifs de réduction de leurs émissions fondés sur la science (50 % en 2025)
-45 % d'émissions de CO₂ liées aux déplacements professionnels et domicile - lieu de travail par rapport à 2019



© Dassault Systèmes | Confidential Information | 2025

Figure 3 : Le modèle d'affaires de Dassault Systèmes

MODÈLE D'AFFAIRES



VALEUR CRÉÉE ET PARTAGÉE

CAPITAL INTELLECTUEL et relation clients

69,8 % chiffre d'affaires éligible à la Taxonomie européenne

25+ ans : durée moyenne de collaboration avec nos 20 principaux clients

Voir les chapitres 1.4.2 et 2.2.2

CAPITAL HUMAIN (collaborateurs)

99 % part des collaborateurs ayant bénéficié de formations

78,4 % taux de fierté et de satisfaction des collaborateurs

98 % part des collaborateurs en CDI

1 400+ offres de stage et d'apprentissage publiées

2 600+ offres d'emploi pourvues en 2024, 95 % en CDI

Voir le chapitre 2.2.3

CAPITAL SOCIAL (société)

279,9 M€ charge d'impôt IFRS (taux effectif d'impôt de 18,9 %)

51 projets d'intérêt général soutenus par la Fondation Dassault Systèmes

10 M+ étudiants utilisant nos solutions **3DEXPERIENCE Edu**

97 % collaborateurs formés à l'éthique et à la conformité

Voir les chapitres 2.2 et 3.1

CAPITAL FINANCIER (actionnaires)

1,28 € bénéfice net dilué par action non-IFRS

Politique de dividende : **30 %** du bénéfice net IFRS

Voir les chapitres 1.7 et 3.1

CAPITAL NATUREL (environnement)

-9 % d'émissions totales de CO₂ par rapport à 2019

85 % part des effectifs dans le monde rattachés à un site certifié ISO pour sa gestion de l'énergie

83 % d'approvisionnement en énergie renouvelable

Voir le chapitre 2.2

Figure 4 : Modèle d'affaires & Valeur créée et partagée

Mission du stage

Contexte

Net to Wire

Un schéma électrique est une représentation logique des connexions physiques ainsi que de la disposition d'un système électrique. Un schéma décrit le fonctionnement d'un circuit électrique et constitue la base pour l'assemblage d'un système électrique.

Dans le cadre de la conception électrique, il existe deux niveaux principaux de granularité des schémas électriques :

Le **Net diagram**, ou schéma fonctionnel électrique, est un schéma qui montre les liaisons électriques entre les équipements, sans considérer comment les fils et les équipements sont placés. La figure 5 montre un schéma d'équipotentielles (nets) :

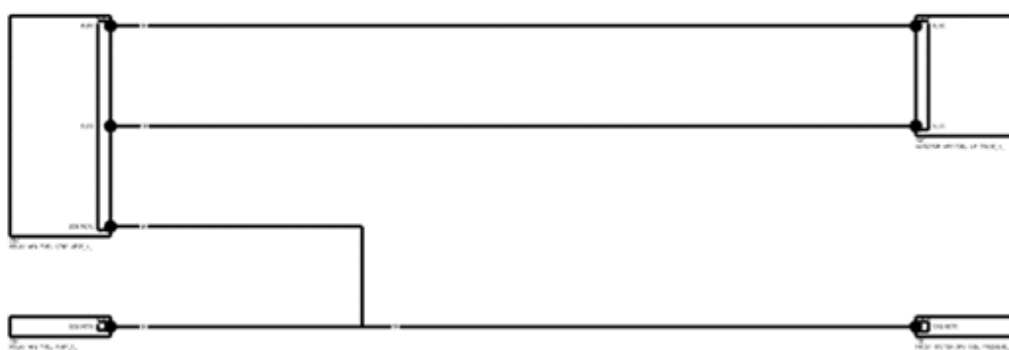


Figure 5 : Net Diagram

Le **Wiring diagram**, ou schéma de câblage, détaille les composants physiques nécessaires à la réalisation du système selon la topologie de l'architecture 3D.

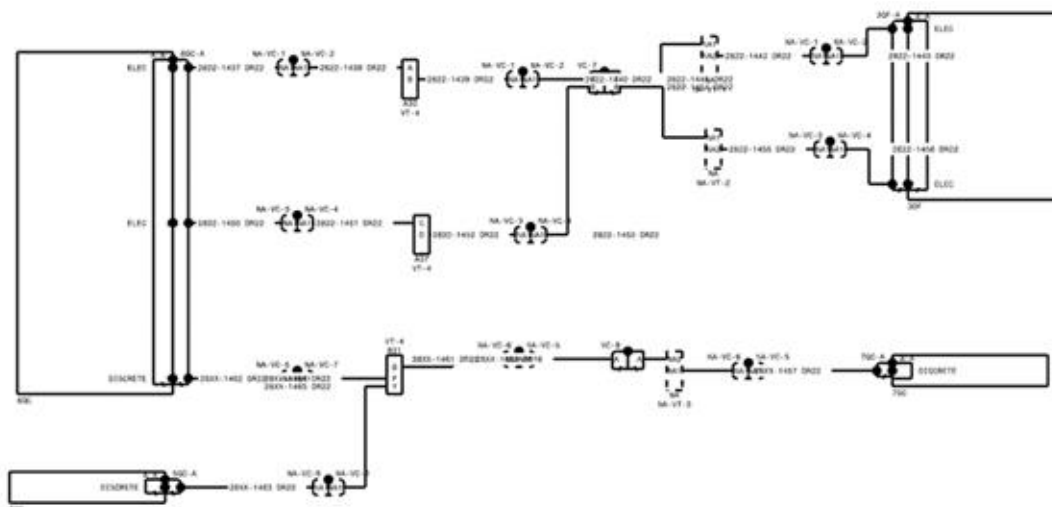


Figure 6 : Wiring Diagram

Ces deux types de schémas représentent deux phases essentielles de la conception électrique. La première phase consiste à établir un schéma fonctionnel électrique, qui définit les éléments électriques et leurs interconnexions. Puis la deuxième phase consiste à préciser l'emplacement des composants électriques et des câbles dans l'environnement physique du système, en tenant compte de sa topologie 3D. En pratique, la transition de la première phase à la deuxième est un véritable défi, car il est nécessaire de prendre en compte l'ensemble des contraintes techniques.



Figure 7 : Exemple de routage dans un hélicoptère / une voiture

Dans ce contexte, l'équipe de CATIA Schématique Electrique propose un outil innovant et puissant : « Net To Wire ». Il permet de faire la transition du Net Diagram au Wiring Diagram, afin d'accompagner les clients dans la réalisation physique de leurs systèmes électriques complexes.

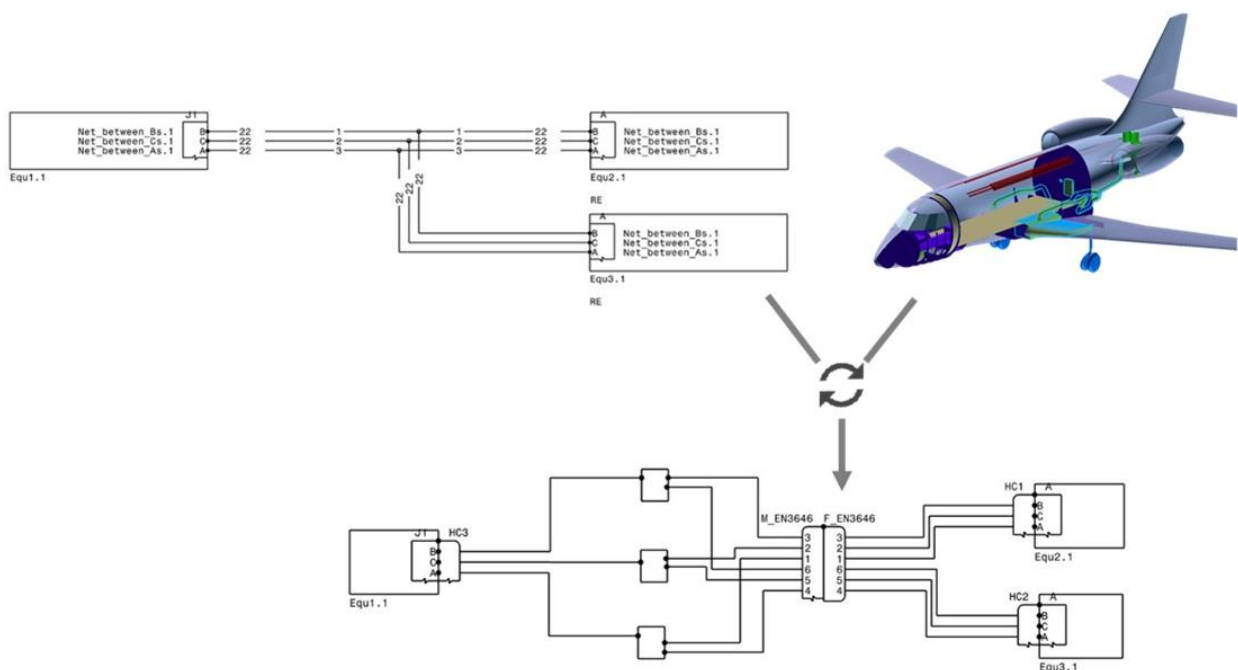


Figure 8 : Net To Wire

Connecteur de coupure et branchement des câbles

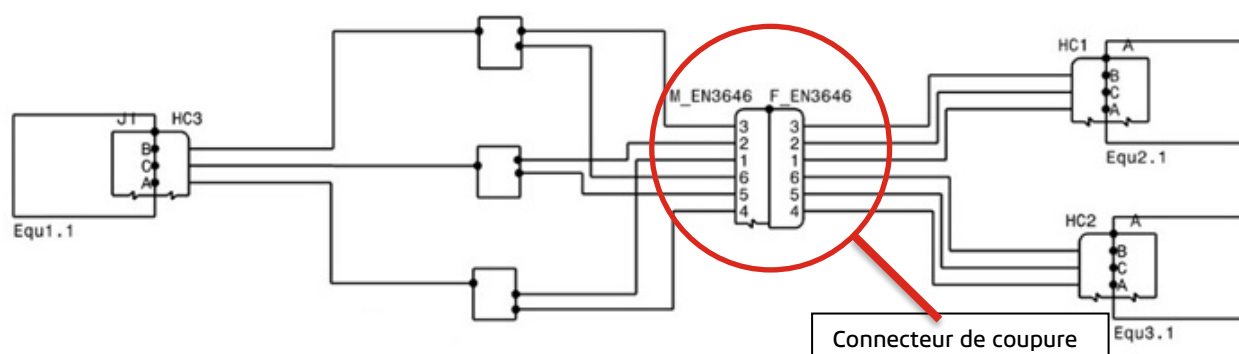


Figure 9 : Connecteur de coupure dans le Wiring Diagram

La figure 9 montre un exemple d'interface de coupure et ses connecteurs associés dans le Wiring Diagram généré par « Net to Wire ». Dans le domaine de l'aéronautique ou de l'automobile, la structure est souvent divisée en plusieurs sous-ensembles. Lorsqu'un câble traverse une coupure structurelle, il doit obligatoirement passer par un **connecteur de coupure**. Les câbles entrants sont connectés aux connecteurs, permettant aux signaux de traverser l'interface et de ressortir de l'autre côté, via des câbles reliés aux connecteurs opposés.

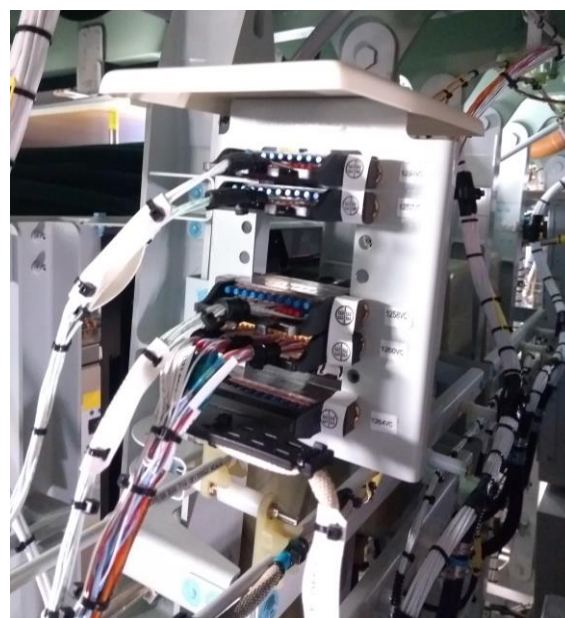


Figure 10 : Photo d'une platine, ses connecteurs et les câbles passants

Objectif du stage

Actuellement, le branchement des câbles dans l'interface de coupure est entièrement réalisé manuellement. Cela présente deux difficultés majeures :

- Premièrement, le nombre de câbles à traiter est important, et leur branchement doit respecter de nombreuses contraintes : compatibilité mécanique, organisation spatiale, séparation des signaux selon les niveaux de ségrégation (les signaux de ségrégation incompatibles ne doivent pas être adjacents), etc. L'affectation manuelle s'avère ainsi particulièrement fastidieuse, chronophage et sujette à des erreurs humaines.
- Deuxièmement, les connecteurs sont souvent partagés entre plusieurs intervenants ou équipes, chacun y branchant des câbles différents. Cette configuration génère fréquemment des situations de conflit, compliquant la coordination et le travail collaboratif.

Ces contraintes opérationnelles soulignent la nécessité de mettre en place une solution plus automatisée, structurée et collaborative pour la gestion des connexions.

L'objectif du stage a donc été de développer une fonctionnalité capable de calculer automatiquement des solutions de branchement et de les proposer aux utilisateurs, afin de simplifier le processus, de réduire les risques d'erreur et de faciliter la coordination entre les différentes équipes.

Au cours de ce stage de six mois, le travail a été organisé autour de cet objectif en trois grandes phases.

- Une première phase algorithmique, qui modélise le problème d'allocation des câbles et détermine une ou plusieurs solutions mathématiques.
- Une deuxième phase de validation, visant à développer une démonstration qui permet d'assurer que l'algorithme conçu est correct et réalisable en pratique.
- Une dernière phase orientée génie logiciel, consistant à implémenter l'algorithme validé en C/C++ avec les infrastructures de Dassault Systèmes, afin de l'intégrer dans l'application CATIA Electrical System Design, en tenant compte des exigences liées à l'environnement industriel.

Ces trois phases seront détaillées dans la suite du rapport.

Phase 1 : Conception d'algorithme

Modélisation de la problématique

Pour pouvoir démarrer ce projet, une modélisation du problème est nécessaire. Après échanges avec l'équipe, la description du problème en langage mathématique est présentée ci-dessous :

Un harnais H est l'ensemble des câbles et connecteurs que nous traitons :

$$H = (C_{\{\text{connecteur}\}}, K_{\{\text{câble}\}})$$

Un connecteur dispose d'une série de cavités, auxquelles on connecte les fils de câbles.

$$C_i = \{P_1, P_2, \dots, P_m\}$$

Où :

- Leur taille R est définie par son diamètre en AWG (American Wire Gauge). Cela détermine la taille de fil acceptée par la cavité.
- Leur coordonnée est donnée par (x, y) . Cela détermine la position de la cavité sur son connecteur parent.



Figure 11 : Un connecteur et ses cavités

Et le câble K_i de la taille R en AWG et de la ségrégation Seg_j , possède une série de fils.

$$K_i = \{W_1, W_2, \dots, W_n\}$$

L'objectif est de trouver une allocation pour chaque câble sur l'ensemble des cavités des connecteurs, en respectant les contraintes suivantes :

- La taille du fil et de la cavité doit être compatibles.
- Les câbles avec des ségrégations incompatibles ne doivent pas être placés sur le même connecteur.
- Les fils d'un même câble doivent être proches les uns des autres.
- Les câbles et leurs fils ne doivent pas se croiser.

La première contrainte est assurée par une vérification de la gauge du fil et de la cavité au moment de la recherche. Pour la deuxième, il suffit de faire un tri par ségrégation du câble au tout début. Pour répondre aux contraintes restantes, il faut encore préciser quelques notions.

La notion de proximité

Les fils d'un même câble doivent être connectés dans des cavités adjacentes. Pour cela, il est nécessaire de définir la notion de proximité et d'établir une table d'adjacence, qui répertorie, pour chaque cavité, toutes les cavités voisines.

La façon la plus simple de définir cette notion, est de tracer un cercle dont le centre est le centre d'une cavité puis de trouver un rayon adapté tel que les cavités adjacentes soient contenues dans ce cercle. Par exemple, ici les cavités 1,3,8 et 9 sont considérées comme cavités voisines de la cavité 2. Cela semble être un moyen simple et efficace.

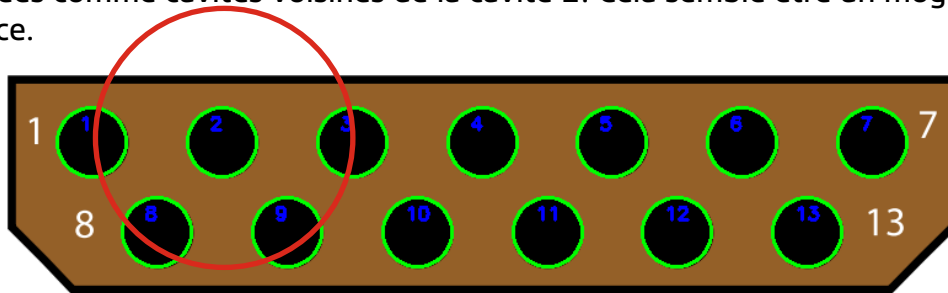


Figure 12 : Connecteur 13 Pins "EN354505" : Correct

Cependant, ce principe montre rapidement ses limites face aux connecteurs complexes, en raison de la difficulté à déterminer un rayon « adapté ». Si ce rayon est défini comme la distance entre une cavité et sa cavité la plus proche, certaines cavités risquent de ne pas être correctement identifiées comme voisines.

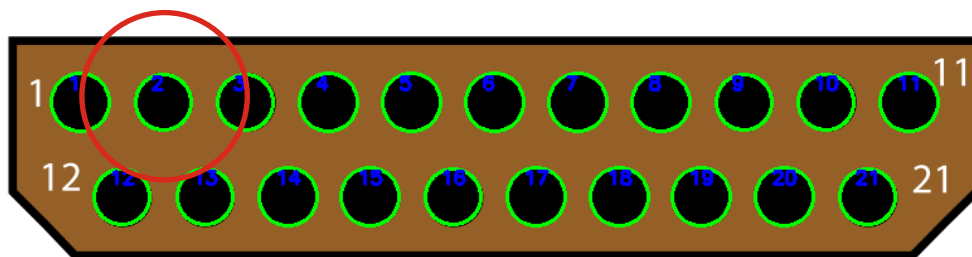


Figure 13 : Connecteur 21 pins "EN354504" : 2 voisines (12 et 13) de la cavité N°2 ignorées

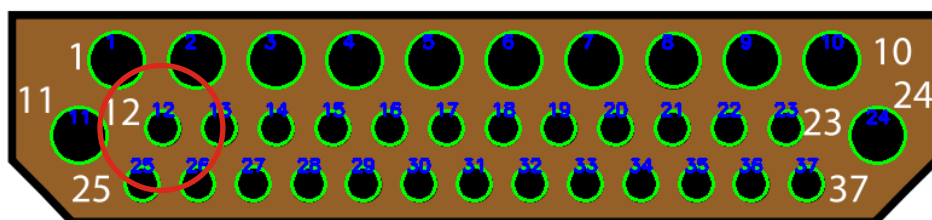


Figure 14 : Connecteur 12 + 25 Pins "EN354509" : Plusieurs voisines (1, 2, 11, 26) de la cavité N°12 ignorées

On observe donc que plusieurs cavités voisines sont ignorées, bien qu'elles soient à côté d'un point de vue intuitif.

Il est possible d'ajouter un coefficient qui donne une marge au rayon pour essayer d'éviter ce problème. Par exemple, dans les deux cas précédents, l'algorithme donnerait un bon résultat si un nouveau rayon $R' = 1,3 R$ était mis en place. Malheureusement, Il n'existe pas de formule permettant de calculer ce coefficient pour un cas quelconque. Finalement, cet algorithme est jugé peu satisfaisant et donc non réalisable.

Inspiré par un projet étudiant réalisé pendant l'UE NF04 (Algorithmique) à l'UTT, une autre approche algorithmique, la triangulation Delaunay, est choisie pour répondre à notre besoin.

La triangulation Delaunay est une structure géométrique construite à partir d'un ensemble discret de points dans le plan $\mathbb{R}^2$. Il s'agit d'une triangulation possédant la propriété suivante :

Pour tout triangle de la triangulation, le cercle circonscrit à ce triangle ne contient aucun autre point de l'ensemble initial.

Cette propriété est appelée « propriété du cercle vide », et elle garantit que la triangulation soit aussi équilibrée que possible. Elle est très utile pour la définition d'une notion de proximité. Par observation, il est facile de constater que dans le résultat de la triangulation Delaunay, deux points connectés par une arête de triangle partagent nécessairement une relation de proximité géométrique.

Cette propriété donne un résultat beaucoup plus satisfaisant que la première méthode. Non seulement parce qu'elle n'ignore jamais des cavités voisine qui ne sont pas les plus proches, mais elle donne aussi un résultat plus intuitif car la notion de proximité construite par la triangulation Delaunay n'est plus basée uniquement sur la distance euclidienne, mais sur la relation spatiale des cavités.

Dans ce résultat de la triangulation, pour chaque cavité, ses cavités voisines sont l'ensemble des autres sommets de tous les triangles de Delaunay qui contiennent cette cavité.

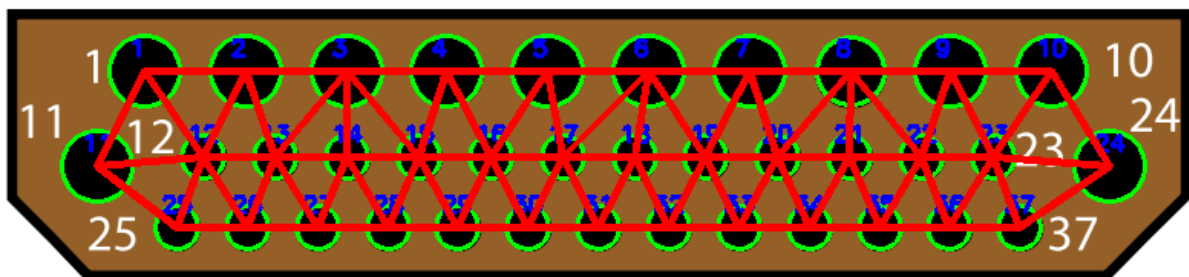


Figure 15 : Connecteur 12 + 25 Pins "EN354509" - Triangulation Delaunay.

Le croisement des câbles

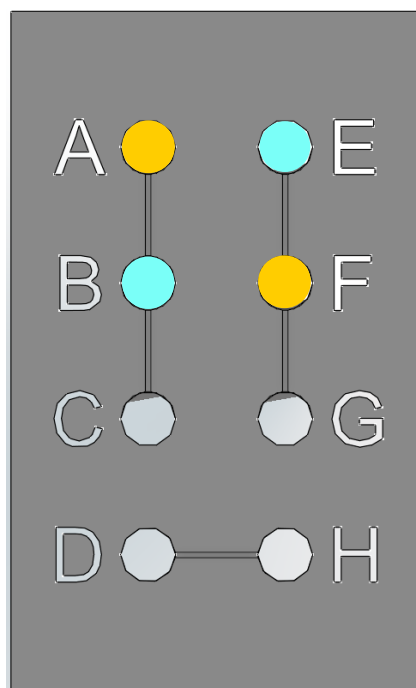


Figure 16 : Croisement des câbles

La figure 16 montre un exemple à éviter. Dans cet exemple, chaque couleur correspond à une zone de cavité allouées à un câble, et un croisement entre deux câbles se produit. Le piège réside dans le fait que ces quatre cavités présentent une répartition carrée, où les deux paires de sommets opposés (AF et BE) sont considérées comme deux paires de cavités voisines. Ainsi, si l'on doit connecter aujourd'hui deux câbles à deux fils, cette solution erronée pourrait être proposée à l'utilisateur. Ce phénomène existe dans les dispositions symétriques (ou presque symétriques). La figure 17 donne un cas de croisement sur une disposition en trapèze :

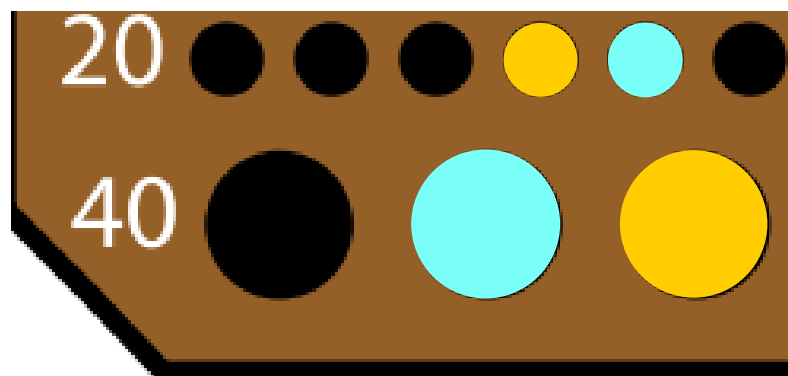


Figure 17 : Un cas gênant sur une partie du connecteur 39 +10 Pins "EN354503"

Néanmoins, ce piège est évité grâce au comportement de la triangulation Delaunay.

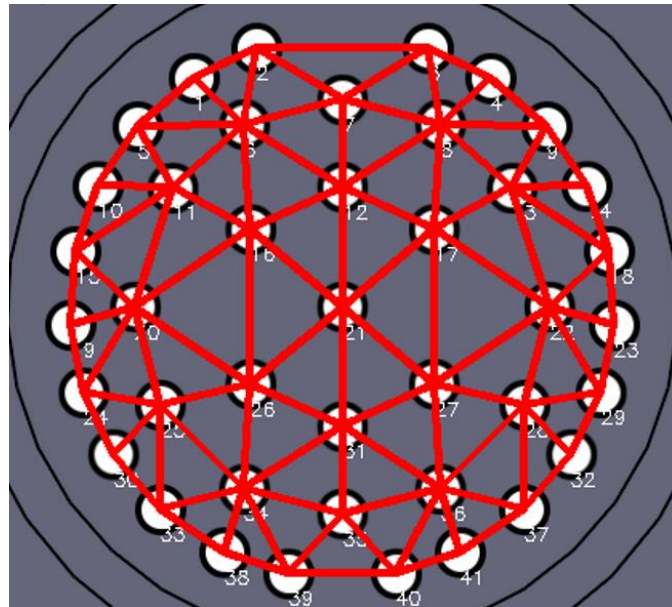


Figure 18: Le résultat de triangulation sur connecteur 41 Pins

Un croisement de câbles peut se produire lorsqu'une ligne reliant deux cavités adjacentes coupe une autre du même type. En tant que méthode de triangulation, elle donne toujours un **graphe planaire**, qui peut se représenter sur un plan sans qu'aucune arête n'en croise une autre. Donc, elle ne génère jamais de triangles croisés et la liste d'adjacence construite à partir de cette triangulation, est donc garantie sans ce type d'intersection.

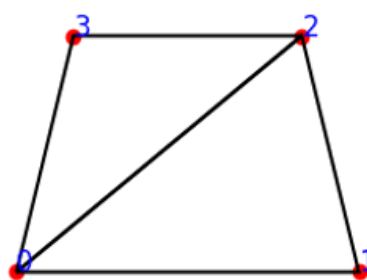


Figure 19 : Résultat de triangulation est garanti planaire.

Ici la cavité 1 et la cavité 3 ne sont pas considérées comme des cavités adjacentes, même si leur distance est identique à celle entre la cavité 0 et la cavité 2. Imaginons un cas où un câble de deux fils a déjà été connecté aux cavités 0 et 2, un autre câble de deux fils, avec un fil placé à la cavité 3, ne pourra jamais placer son deuxième fil à la cavité 1. Le croisement des câbles est donc évité.

Stratégie de recherche

La recherche d'allocation pour chaque câble est basée sur la table d'adjacence des cavités établie dans l'étape précédente. Le processus de recherche est un processus de parcours d'arbre : la racine est la cavité de départ, les cavités voisines de chaque nœud sont les nœuds enfants, et la profondeur de recherche correspond au nombre de fils dans un câble.

La méthode la plus simple pour trouver toutes les solutions possibles, est sans doute la recherche par retour sur trace, dite « Backtracking » en anglais. C'est une approche algorithmique très souvent utilisée pour résoudre des problèmes de recherche combinatoire. Elle explore l'ensemble des solutions possibles sur un parcours en profondeur (DFS).

Néanmoins, bien qu'elle ait une simplicité de réalisation, elle a une complexité temporelle trop importante dans certain cas pratique. Pour une démonstration du problème, faisons une preuve simple par récurrence pour déterminer la complexité temporelle de cet algorithme.

Initialisation :

Soit b le nombre de nœuds enfants pour chaque nœud.

Si la profondeur de recherche $d = 0$, seulement la racine est visitée.

$$T(0) = 1 = O(b^0) = O(1)$$

Hérédité :

Supposons qu'il existe une constante $c > 0$ telle que, pour une profondeur quelconque $k \in \mathbb{N}$, le nombre maximal d'étapes explorées est de :

$$T(k) \leq c \cdot b^k$$

Considérons une exploration de profondeur $k + 1$, Pour chaque choix, la suite de l'exploration correspond à une sous arborescence de profondeur k . dont le nombre d'étapes est $c \cdot b^k$. La totalité de l'exploration de profondeur $k + 1$ est de :

$$T(k + 1) \leq 1 + b \cdot T(k) = 1 + b \cdot c \cdot b^k = 1 + c \cdot b^{k+1}$$

Avec la notation big O, nous obtenons la complexité temporelle de cet algorithme :

$$O(b^d)$$

Dans un scénario d'une allocation de 20 fils sur 20 cavités, donc un cas assez raisonnable en pratique, avec un connecteur d'une distribution en forme de grille hexagonale (type "nid d'abeille"), où chaque cavité est à côté de 6 cavités voisines, on compte 6^{20} possibilité ce qui est approximativement égal à $3,66 \cdot 10^{15}$. Si C++ permet de faire 10^6 calculs chaque seconde, cette opération d'allocation automatique prendra **116 ans**.

Un autre problème est le nombre de solutions proposées. Voici une petite expérience pour estimer le nombre de solutions valides pour un certain nombre de fils au total.



Figure 20 : Connecteur "EN354503", 39 Pins 22

Ce tableau montre le nombre de solutions sur le connecteur 39 pin « EN354503 » :

Profondeur de recherche	Nb. de solution valide
2	75
4	5 114
6	314 970
8	17 394 840

Lorsque le calcul de profondeur = 10, la mémoire utilisée par l'algorithme dépasse 32 Go. On ne peut donc pas avoir le résultat exact. Mathématiquement, le nombre de résultats dans ce cas est d'environ 850 000 000.

D'après ces raisonnements et démonstrations, bien que cet algorithme permette théoriquement de trouver toutes les solutions valides, son coût en pratique est inacceptable. Cela rend son usage peu réaliste sans une stratégie de réduction de l'espace de recherche. De plus, il est inutile de consacrer autant de temps à chercher toutes les solutions possibles car les utilisateurs ne sont pas en mesure de gérer des dizaines ou des centaines de millions de solutions proposées.

L'orientation du projet a été rapidement revue après avoir constaté que la première version n'était pas réalisable. Une autre approche est d'essayer de trouver quelques solutions qui sont meilleures que les autres. Une fois qu'une solution qualifiée est trouvée, le programme arrête le calcul et la propose aux utilisateurs. C'est une méthode plus réaliste et qui s'adapte mieux aux besoins des clients.

La notion de « meilleure solution » d'un câblage est ambiguë. Cependant, au regard des contraintes définies lors de la phase de modélisation, il est tout à fait possible d'écarter certaines solutions manifestement moins pertinentes.

Cette solution génère des cavités isolées (1 et 14). Avec les évolutions futures des systèmes, il est très probable que de nouveaux câbles soient ajoutés à l'avenir, mais ces cavités isolées ne pourront pas être exploitées.

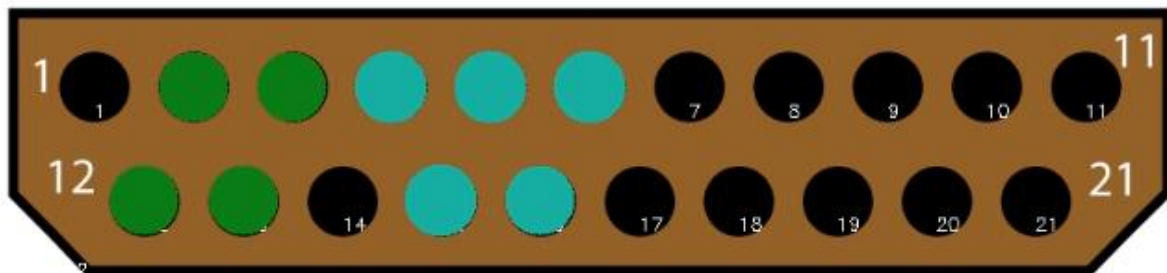


Figure 21 : Solution problématique : Génération de cavités isolées

1. Il vaut mieux de regrouper les câbles sur des cavités adjacentes, pour faciliter la gestion et pour éviter une allocation chaotique, désorganisée qui pourrait dégrader la lisibilité et rendre la maintenance plus difficile.

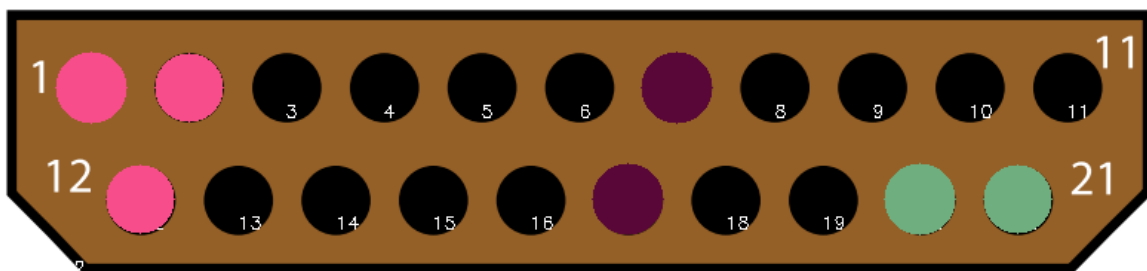


Figure 22 : Solution problématique : Répartition des câbles trop dispersée

2. La figure 23 montre trois stratégies de placement. De manière intuitive, une organisation centralisée (cas à gauche) est considérée comme meilleure qu'une répartition linéaire (cas au milieu) car elle favorise la compacité. L'idée est de concevoir un indice permettant d'évaluer le niveau de centralisation des cavités allouées, puis de laisser l'utilisateur choisir la répartition qu'il souhaite.

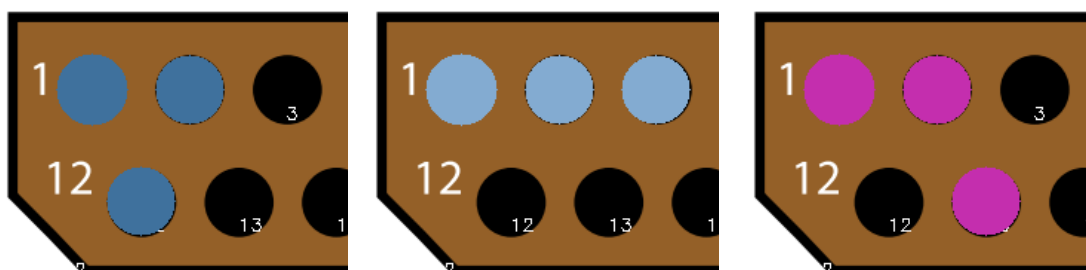


Figure 23 : Solutions valides avec différentes formes

En tenant compte de ces cas, le nombre de solutions est considérablement réduit.

Phase 2 : Implémentation d'un prototype

Dans la première phase, une version fonctionnelle de l'algorithme a été conçue. Bien qu'elle offre encore de nombreuses possibilités d'amélioration en termes de complexité temporelle, de précision, etc., elle est capable de résoudre la problématique.

Dans une deuxième phase, un prototype en C/C++ standard est réalisé dans le but de vérifier la performance, la précision, et le comportement global de l'algorithme. Cela a permis de se concentrer sur la logique algorithmique sans être bloqué par les contraintes techniques de la plateforme.

Il facilite également le débogage et l'identification des pistes d'amélioration avant une éventuelle intégration finale dans CATIA, qui se révèle plus complexe.

Ce prototype est réalisé avec Ubuntu 24.04(wsl2), Python 3.12, GCC 14.2 et CMake 3.28. L'avantage d'utiliser le standard C++23 est de profiter des technologies et des outils les plus puissants et les plus récents.

La partie python consiste à automatiser la saisie de données et à visualiser le résultat. La communication entre C++ et python se fait par pybind11.

Auto-détections des cavités

Ce processus est composé d'un script python « cavity_detector ». Son rôle est d'identifier toutes les cavités et d'enregistrer leur position et leur rayon à partir d'un fichier d'image du connecteur.

Il utilise la fonction « findContours » de la bibliothèque OpenCV pour trouver les cavités, qui sont graphiquement des cercles, avec quelques traitements préalables d'image.

Des traitements comme un renforcement du contraste de l'image ou un filtre avec un masque de couleur sont nécessaire avant de scanner les contours pour un meilleur résultat de détection.

Un contrôle de circularité est effectué sur les cercles trouvés, pour éviter les fausses détections, notamment les lettres qui ressemblent à un cercle, comme « C » ou « G ».

La correspondance entre le rayon et le gauge est saisie à la main. Les données détectées, sous la forme d'une liste de tuples [ID, gauge, x, y], vont être envoyées à l'exécutable C++ grâce à la bibliothèque pybind11, pour les calculs.

Nous avons pensé à une solution basée sur l'intelligence artificielle, mais elle a été rapidement jugée non faisable en raison du manque de données d'apprentissage.

Conception de la structure de données

Deux types d'objets de base sont définis : le fil (Wire) et la cavité (Cavity), chacun identifié par un ID unique et une taille normalisée (en AWG).

Pour garantir que chaque objet soit bien unique et éviter les erreurs liées à des duplications non contrôlées, la copie de ces objets a été rendue impossible. Ils ne sont manipulables que via des pointeurs intelligents (`std::shared_ptr`), ce qui permet un meilleur contrôle de leur durée de vie et de leur unicité.

Une interface commune pour ces deux classes (`electronic_component_base`) permet de vérifier leur disponibilité et leur compatibilité.

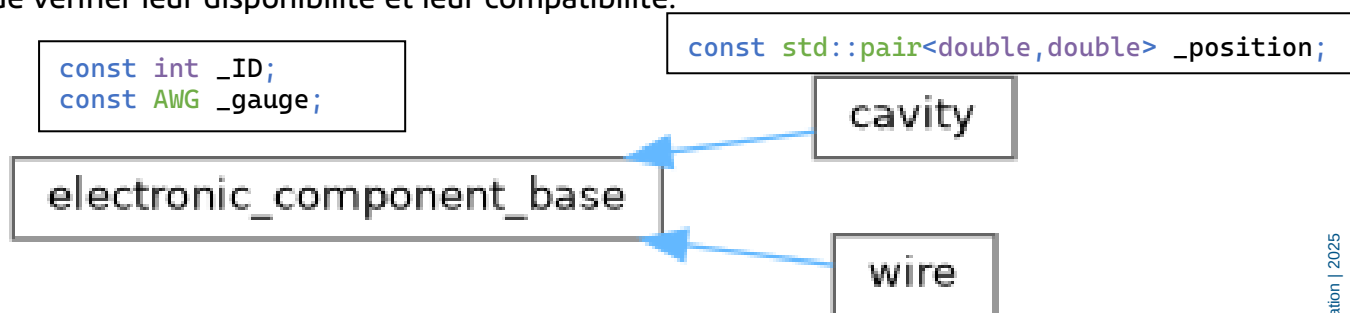


Figure 24 : Relation des classes et l'interface de composant

Les classes de câble et de connecteur implémentent une interface de type conteneur (`electronic_container_base`) capable de gérer une liste de composants. Le connecteur calcule à son initialisation une table d'adjacence en utilisant la triangulation Delaunay. Le câble, de son côté, conserve un ensemble pour enregistrer les solutions d'allocation possible et un pointeur qui pointe vers le connecteur auquel il est connecté.

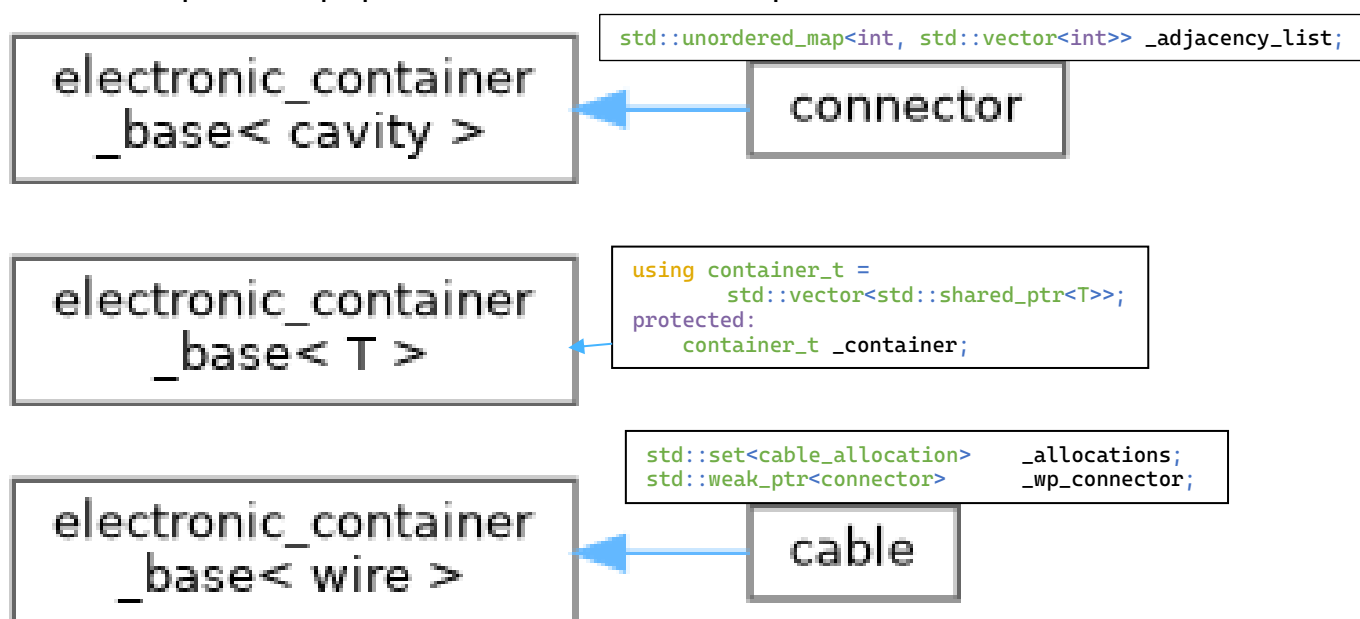


Figure 25 : Relation des classes et l'interface de

Triangulation Delaunay

Afin de déterminer quelles cavités sont voisines dans un connecteur, une méthode de triangulation appelée Bowyer-Watson a été utilisée.

Une liste des triangles représente les triangles dans la triangulation. Au départ, on crée un grand triangle imaginaire qui englobe toutes les cavités et on l'ajoute dans la liste. Ensuite, les cavités sont ajoutées une par une dans le graphe. À chaque insertion, on vérifie si la cavité ajoutée est dans le cercle circonscrit de chaque triangle dans la liste des triangles. Si oui, ce triangle est marqué « mauvais » est alors supprimé. On utilise la cavité nouvellement ajoutée et les trois arêtes du triangle supprimé pour reconstruire trois nouveaux triangles, qui vont être ajoutés dans la liste des triangles.

Une fois toutes les cavités insérées, on supprime les triangles contenant les sommets du triangle imaginaire initial. Les triangles restants constituent une triangulation Delaunay. Le code de cet algorithme est dans l'Annexe 2.

Recherche DFS

L'algorithme DFS pour le parcours d'arbre est simple : pour chaque câble, on commence par une cavité disponible (la racine), puis on parcourt ses nœuds enfants en vérifiant à chaque fois la compatibilité entre le fil et la cavité. Une fois la profondeur maximale atteinte (c'est-à-dire, tous les fils du câble sont affectés), on enregistre le résultat comme une solution valide. Un retour sur trace est effectué ensuite pour explorer d'autres possibilités. Voici un extrait du code de l'algorithme :

```
for (const auto& cavity : searching_range)
{
    std::vector<int> visited_cavity_indices;
    auto dfs =
        [&](self, cavity_ID, wire_ID)
        {
            if (// Cavity is not available or compatible) return;

            visited_cavity_indices.push_back(cavity_ID);
            if (visited_cavity_indices.size() == // Nb of wires in cable)
                // All wires are allocated, Save valid result.
                _allocations.emplace(visited_cavity_indices);
            else
                for (// All current cavity's neighbor)
                    if (// Neighbor cavity is not visited)
                        self(self, neighbor, wire_index + 1);
            // Backtrack
            visited_cavity_indices.pop_back();
        };
    dfs(dfs, cavity->get_ID(), 0);
}
```

L'évaluation d'une solution

La stratégie de recherche impose une allocation câble par câble. Or, dans la pratique, cette recherche génère souvent un grand nombre de solutions valides pour un même câble. Pour cette raison, un mécanisme d'évaluation a été introduit afin d'évaluer la qualité de chaque solution et de proposer la plus pertinente à l'utilisateur.

Dans ce prototype, plusieurs solutions sont affichées avec leur score respectif, ce qui permet de valider le bon fonctionnement du système d'évaluation. À terme, dans la version finale, seule la meilleure solution sera proposée à l'utilisateur.

Une solution pour un câble correspond à une zone de cavités occupée par ce câble. Pour simplifier, la version actuelle utilise un critère unique : le rayon du plus petit cercle englobant, calculé avec l'algorithme de Welzl. Dans un contexte industriel, ce score pourra devenir un critère composite tenant compte de plusieurs paramètres techniques, personnalisables selon les besoins des clients.

Les solutions valides triées par leur score sont affichées dans la console. La figure 26 est le résultat qui propose plusieurs solutions pour un câble à trois fils. Le programme invite l'utilisateur à sélectionner une solution en tapant le numéro correspondant.

```
Cable 3 - Allocation 0, Score: 56
cavity 6
cavity 7
cavity 13
Cable 3 - Allocation 1, Score: 56
cavity 6
cavity 12
cavity 13
Cable 3 - Allocation 2, Score: 88
cavity 6
cavity 7
cavity 12
Cable 3 - Allocation 3, Score: 88
cavity 7
cavity 12
cavity 13
Choose allocation index for cable 3: █
```

Figure 26 : Liste des solutions proposée à l'utilisateur

Visualisation

Le script python « visualizer » prend les données envoyées par la partie C++, qui indiquent la solution choisie, et génère une visualisation correspondante. Pour distinguer les différents câbles, les cavités occupées par différents câbles sont affichées en différentes couleurs distribuées aléatoirement.

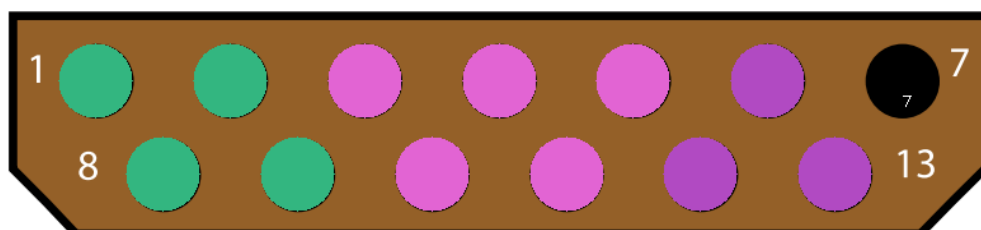


Figure 27 : Exemple d'une visualisation

Phase 3 : Implémentation d'une commande CATIA

L'intégration finale du prototype dans CATIA s'est déroulée de manière fluide. L'algorithme ainsi que son implémentation issue du prototype ont été intégralement conservés. Cela a été possible grâce aux anticipations faites dès la phase de prototypage. Cependant, il existe certaines différences entre le C/C++ standard et celui de Dassault Systèmes ont nécessité des adaptations.

Introduction d'infrastructure

Jusqu'à CATIA V4 (1993), le langage de programmation chez Dassault Systèmes était le FORTRAN, le langage n°1 dans le domaine du calcul numérique. Malgré sa performance, il ne parvient plus à s'adapter aux exigences organisationnelles des grands projets de logiciels. Pour répondre à la croissance rapide du nombre de modules et de développeurs, CATIA V5 est devenu un projet en C++, avec une version propriétaire d'architecture COM de Dassault Systèmes, appelée « **Object Modeler** ».

Dans Object Modeler, l'objet de base s'appelle un « **component** ». Il a obligatoirement une **implémentation**, qui gère ses propriétés de base, et une ou plusieurs **extensions**, qui permettent de lui ajouter d'autres fonctionnalités. La cycle de vie d'un component est gérée par un compteur de référence interne. Quand le nombre de référence atteindre 0, le component est détruit.

Les fonctions d'un composant sont définies par une ou plusieurs **interfaces**. Ces classes abstraites ne contiennent que des méthodes virtuelles pures, chacune correspondant à une opération ou un service que le composant s'engage à fournir. L'importance de cette séparation entre l'interface et l'implémentation se révèle importante au fur et à mesure que le projet grandit, car elle permet aux développeurs de travailler sur des secteurs différents : certains se concentrent sur la signature définie par l'interface, d'autres sur l'algorithme concret ou l'optimisation de performances dans la classe d'implémentation.

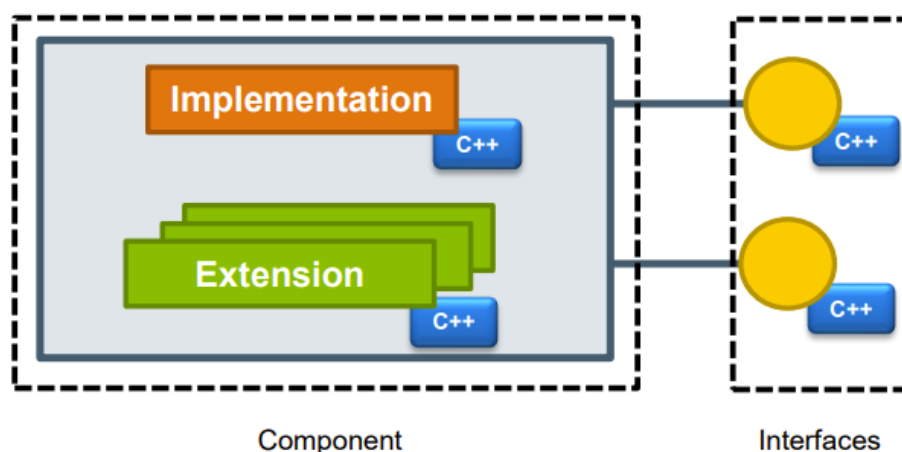


Figure 28 : Modèle d'Object Modeler

Solution existante

Aujourd'hui, l'équipe CATIA Schématique Electrique propose déjà l'outil de « Wiring Definition Assistant ». Cet assistant permet de connecter les câbles dans les connecteurs de coupure en se basant sur le routage 3D généré par l'outil « Net to Wire ». Il est donc intégré dans la section « Net to Wire » de CATIA Electrical System Design.

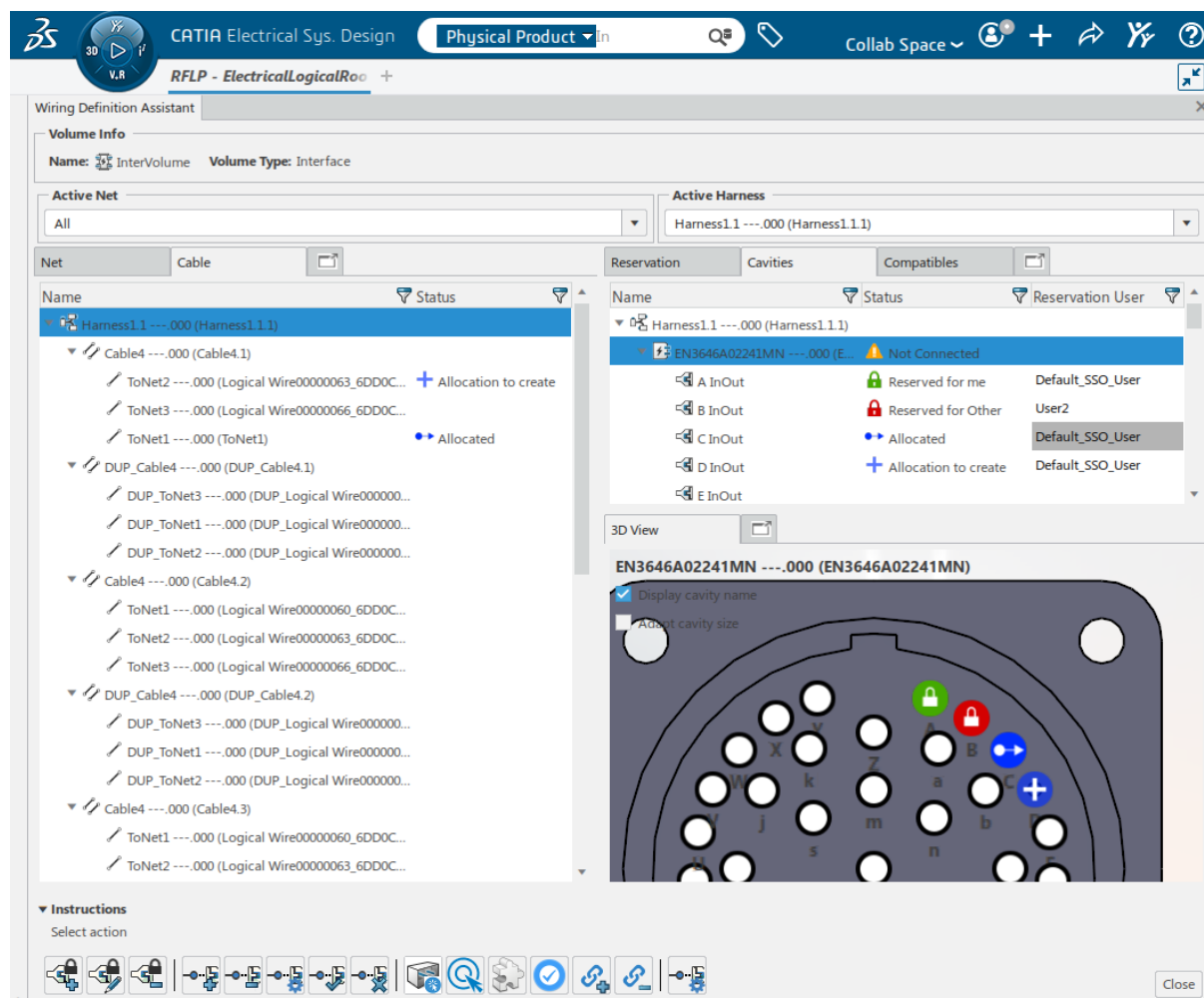


Figure 29 : Wiring Definition Assistant

Dans ce panneau, les éléments sont organisés comme suit :

- À gauche : la liste des câbles et de leurs fils.
- À droite : la liste des connecteurs de coupure et de leurs cavités, ainsi que la vue 3D de la pièce physique correspondante.
- En haut : deux filtres sont disponibles :
 - Active Net : filtrer les câbles d'une liaison équipotentielle.
 - Active Harness : filtrer les câbles & connecteurs du même harnais.
- En bas : plusieurs boutons sont présents dans la barre d'outils. Ils permettent de créer, de confirmer et de supprimer une réservation de cavité ou une allocation entre fils et cavités.

Le mécanisme de réservation de cavité est utilisé pour éviter les conflits d'allocation lorsque plusieurs utilisateurs interviennent simultanément sur la même interface.

Les statuts sont visibles dans la colonne "Status" ainsi que dans la vue 3D du connecteur. Les statuts possibles et leurs icônes associées sont les suivants :

- Free (Libre) : Pas d'icône spécifique.
- Reserved for me (Réservé pour moi) : Icône verte avec un serrure.
- Reserved for Other (Réservé pour un autre) : Icône rouge avec un serrure.
- Allocation to create (En attente de confirmation) : Icône bleue avec un "+".
- Allocated (Allouée) : Icône bleue avec une flèche "->".

Ces éléments existants fonctionnent correctement, mais le processus de réservation et d'allocation repose entièrement sur des manipulations manuelles pour chaque cavité, ce qui est lent et peu efficace. De plus, la conception de l'interaction utilisateur pourrait également être améliorée. Certaines opérations simples demandent plusieurs étapes successives, ce qui alourdit encore le processus.

Création d'une nouvelle commande CATIA

La nature des fonctionnalités présentées dans le panneau « Wiring Definition Assistant » est une commande CATIA. Pour ajouter la nouvelle fonctionnalité d'allocation automatique, il faut créer une nouvelle commande. La nouvelle commande, nommée `CATEditorCableToConnectorAllocation`, sera désignée par la suite simplement comme **la commande**.

Pour créer une nouvelle commande, il faut d'abord l'enregistrer sous la forme de « CommandeHdr » dans le fichier « ModuleHeaders.cpp ». Dans notre cas, nous enregistrons le header de la commande dans « CATEditorHeaders.cpp »

Ensuite, on implémente la commande dans le fichier « CommandeCmd.cpp ». Un pointeur qui pointe sur le panneau est requis lors de l'initialisation de la classe de commande pour la communication. `BuildGraph()` est un membre indispensable qui définit l'algorithme de la commande. La fonction `AddTransition` permet d'ajouter une étape au algorithme. Avec cette fonction, les états du dialogue (`CATDialogState`), les conditions à vérifier et les fonctions à appeler lors de la transition des états sont définis. Voici sa signature :

```
virtual CATDialogTransition * AddTransition (CATDialogState* iSource,
                                             CATDialogState* iTarget,
                                             CATStateCondition* iCondition,
                                             CATDiaAction* iAction);
```

Cette fonction permet à la commande de passer d'un état(`iSource`) à un autre(`iTarget`), en vérifiant la condition(`iCondition`) et en appliquant l'action (donc la fonction à appeler lorsque la condition est satisfaite).

L'interface utilisateur et l'introspection

Pour que la commande soit accessible à l'utilisateur, il faut la mettre dans l'interface graphique (GUI). Lors de la conception de l'interface utilisateur, il est nécessaire de mettre en place un style visuel homogène avec les fonctionnalités déjà existantes dans le panneau.

La programmation d'UI est différente de la programmation « back-end ». Une capacité indispensable pour la programmation d'UI est la capacité d'introspection. C'est une capacité de pouvoir observer dynamiquement(en runtime) les attributs et les caractéristiques d'un objet lui-même. Étant un langage de programmation statique, C++ n'a pas cette capacité.¹ Mais, Dassault Systèmes a mis en œuvre un mécanisme similaire qui s'appelle CID (Common Interface Design). Il génère un fichier listant les propriétés des classes, ainsi qu'un code permettant l'introspection et l'accès à la valeur d'une propriété à partir de son nom.

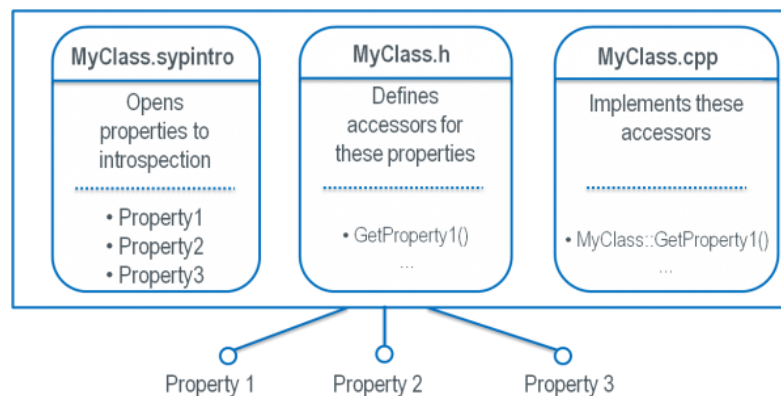


Figure 30 : Model d'une classe C++ introspectable

Création d'un bouton « CableAllocationButton » dans un fichier .sypstyle :

```
<CATVidCtlButton    syp:name="CableAllocationButton"
                    Name="CableAllocationButton"
                    IconDimension="32 32"
                    Icon="I_CATELEditorAllocationAutomatic"
                    Click="@OnCable2ConnectorAllocationClick"
                    syp:Template="Button"/>
```

On voit bien les caractéristiques : son nom, taille, icône et l'association du bouton à son événement « clic ». Cet événement « OnCable2ConnectorAllocationClick » est défini comme un **EventHandler** dans le fichier .sypintro :

```
<EventHandler        name="OnCable2ConnectorAllocationClick"
                    args="CATSYPEventArgs"/>
```

¹ Une proposition [P1240R2] a introduit l'introspection et la réflexion statique dans C++26, mais elle n'est pas encore officiellement finalisée ni intégrée dans la norme.

Dans le header du panneau, on enregistre ce bouton avec un pointeur. A l'aide de la fonction `RetrieveSypNamedObject()`, ce poineur va récupérer le bouton que nous avons défini dans le fichier `.sypintro`.

L'événement `OnCommandClick`, déclenché par un clic gauche du bouton, est défini aussi dans la classe du panneau. Pour lancer la commande, il suffit de récupérer son header qui a été enregistré dans le « `CATELEditorHeaders.h` ». Ce header récupéré a une méthode qui s'appelle `StartCommand()` pour lancer la commande associée.

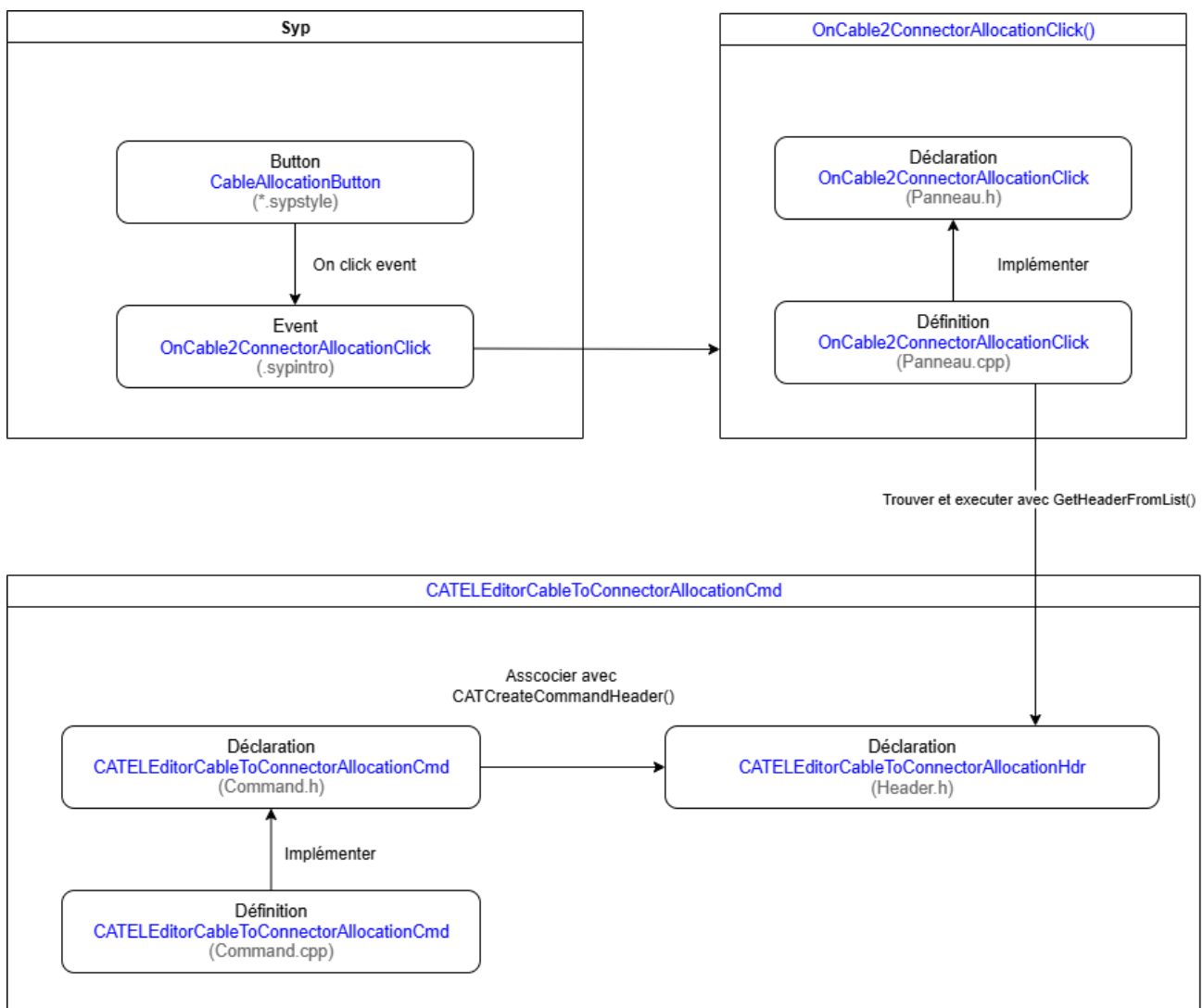


Figure 31 : Mode de fonctionnement de la commande

Nouvelle classe de cavité

Dans le contexte de CATIA Electrical System Design, on distingue la notion d'objet « logique » de celle d'objets « physique ». Chacun possède sa propre définition et son propre comportement. Cependant, ces deux types d'objet portent le même nom, ils sont donc reliés par ce nom en commun. Notre projet d'allocation automatique est un projet mixte entre le niveau « logical » et le niveau « physical », il a besoin de ces deux types d'objets de cavité. En effet, la cavité logique est indispensable pour consulter sa disponibilité, récupérer ses propriétés et créer sa connexion avec le fil. La cavité physique connaît ses coordonnées « 3D », ce qui est crucial pour le calcul de la liste d'adjacence. La création d'une nouvelle classe de cavité, qui réalise la combinaison des toutes ces caractéristiques, est donc nécessaire.

L'approche la plus simple et intuitive est de garder un pointeur pour chaque type d'objet. Avec ces deux pointeurs, il est toujours possible de consulter les propriétés souhaitées ou faire des modifications. Néanmoins, elle comporte un inconvénient : nous ne sommes pas responsables des objets pointés. Utiliser un smart pointeur pour un objet dont on ne possède pas la propriété est très dangereux, car cela introduit une ambiguïté dans la gestion de la durée de vie de l'objet. L'objet pourrait ne pas être supprimé au bon moment car le compteur de références reste à 1 du fait de la présence de ce pointeur. Cette situation peut entraîner des fuites mémoire ou d'autres comportements indéfinis.

À l'inverse, avec des pointeurs classiques, si l'objet pointé a déjà été détruit au moment de l'utilisation du pointeur, le programme provoquera une erreur de segmentation ou une violation d'accès mémoire.

Une autre approche plus pratique est de récupérer et copier toutes les propriétés nécessaires des deux objets lors de l'initialisation. De cette façon nous possédons nous-mêmes les informations dont nous avons besoin et nous ne nous inquiétons plus du problème de cycle de vie et d'utilisation de pointeur invalide. Elle améliore aussi la performance en limitant les appels fréquents aux fonctions d'accès, qui peuvent coûter très cher pour un objet dans CATIA.

Chaque objet CATELEditor3DCavity, nouvellement créé, est stocké dans une table d'association par nom. Vu que toutes les cavités d'un même connecteur sont sur le même plan, et que notre algorithme de triangulation Delaunay est conçu pour le cas 2D, il faut faire une réduction de dimensionnalité pour supprimer la troisième dimension inutile. CATIA propose des outils géométriques internes simples et puissants. Les trois premières cavités sur le connecteur sont choisies pour construire un plan, puis les cavités restantes seront projetées dans ce plan. Après cette opération, les cavités seront envoyées à la fonction qui réalise la triangulation Delaunay, pour la construction de la liste d'adjacence de cavités dans le connecteur actuel.

Résultat

Un bouton pour la commande « Automatic Allocation » est ajouté dans la barre d'outils du panneau.

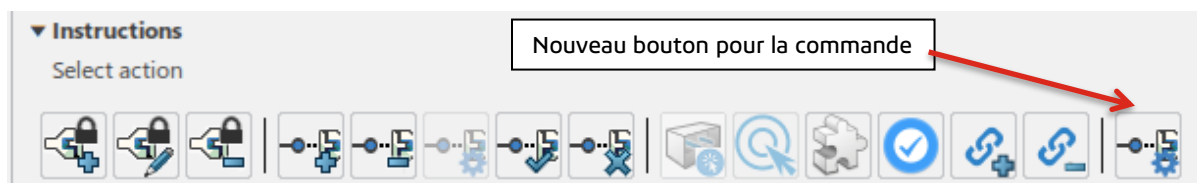


Figure 32 : Position de bouton dans la barre d'outils

En cliquant sur ce bouton, tous les câbles à connecter dans le harnais actif sélectionné vont être connectés au premier connecteur disposant encore de cavités disponibles (donc en état « Free » ou « Reserved for me »). Une fois qu'un connecteur est entièrement rempli, le processus continue avec le prochain connecteur. De plus, les câbles appartenant à des groupes de ségrégation différents sont automatiquement connectés à des connecteurs différents, afin de garantir l'isolation physique entre ces groupes.

Figure 34 correspond à la solution proposée à l'utilisateur. Elle assure que les cavités allouées au même câble sont adjacentes et évite l'apparition de cavité libre isolée. Si cette allocation ne lui convient pas, l'utilisateur peut l'ajuster en détail avec les commandes existantes selon sa préférence.

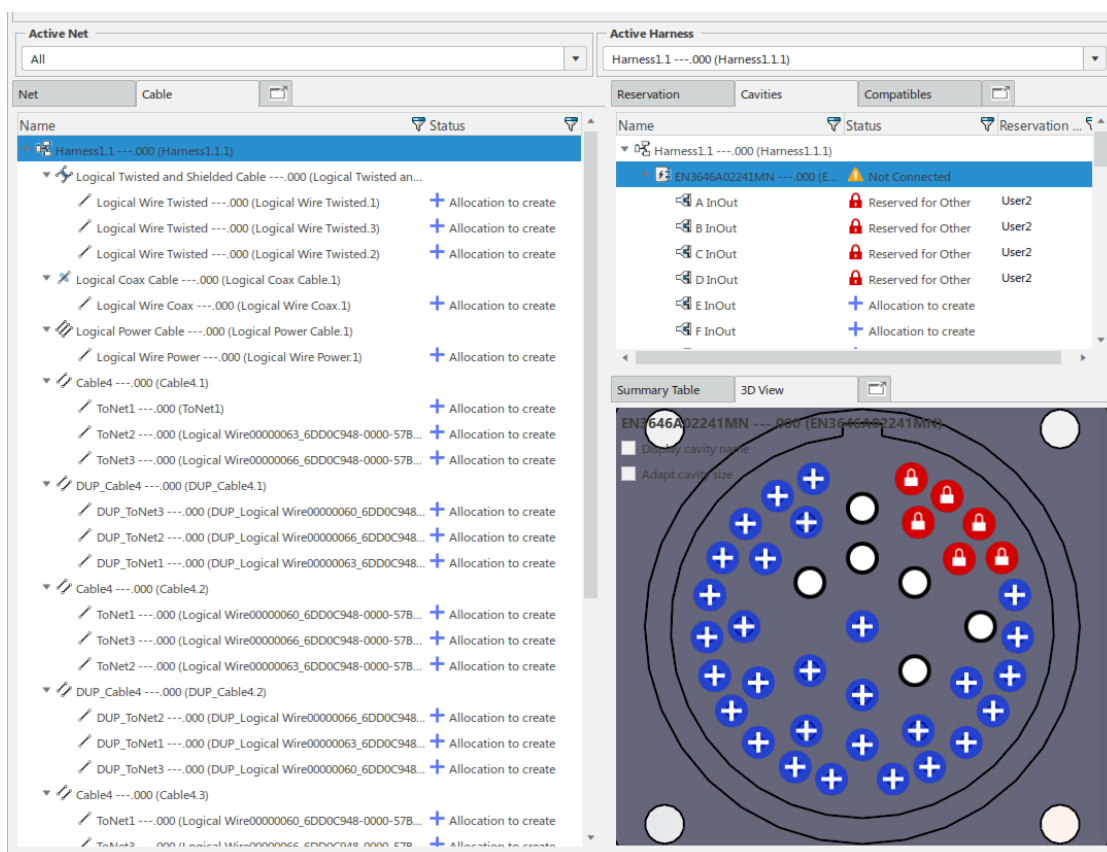


Figure 33 : Allocations en attendant de validation par l'utilisateur

Bilan technique et professionnel

Ce stage de fin d'études, réalisé au sein de l'équipe de CATIA Schématique Electrique, m'a permis de m'immerger dans un environnement industriel structuré et stimulant. La mission du stage, l'automatisation d'allocation de câbles aux connecteurs de coupure, m'a offert une vision d'ensemble du processus de développement logiciel en grande entreprise : de la modélisation du besoin jusqu'à l'intégration d'une commande dans l'environnement déployé.

Au fil du stage, j'ai pu utiliser mes connaissances en algorithmique, en résolution du problème, ainsi qu'en développement logiciel pour atteindre l'objectif. Ce bilan revient sur les principaux enseignements que j'en ai tirés, aussi bien sur le plan technique que sur le plan humain et professionnel.

Sur le plan technique, j'ai eu l'occasion :

- De concevoir un algorithme pour réaliser une allocation automatique de câble sur des connecteurs de coupure, en répondant les besoins clients et en respectant des contraintes.
- D'explorer et d'appliquer des techniques d'optimisation géométrique, comme la triangulation de Delaunay et l'algorithme de Welzl pour le cercle englobant minimal, afin de formaliser la notion d'adjacence spatiale entre cavités et évaluer une solution.
- De mettre en œuvre un prototype avec l'algorithme de recherche (un parcours d'arbre) de retour sur trace, en analysant rigoureusement sa complexité, et en identifiant les cas non valides ou non pertinents.
- De faire l'adaptation de l'algorithme à la plate-forme 3DEXPERIENCE CATIA, un environnement de développement complexe, dans lequel les outils modernes coexistent avec des infrastructures historiques non basées sur la C/C++ standard.

Dans ces étapes du stage, j'ai pris conscience que le développement logiciel en contexte industriel exige bien plus que des algorithmes performants ou une utilisation élégante du langage.

La robustesse, la lisibilité du code, la réutilisabilité et la compatibilité avec l'existant priment souvent sur l'innovation technique pure ou l'optimisation marginale. J'ai également expérimenté l'importance du travail collaboratif, de l'écoute des besoins utilisateurs et de la progressivité dans la mise en œuvre des solutions, notamment en intégrant mes développements dans une chaîne d'outils existante.

Ce stage m'a permis d'observer de près le fonctionnement d'un logiciel industriel de très grande échelle comme 3DEXPERIENCE. Avant cette expérience, J'étais curieux de comprendre comment un produit aussi massif pouvait être conçu et maintenu. J'ai découvert une organisation rigoureuse, répartie à l'international, avec des milliers de développeurs travaillant dans des équipes spécialisées et souvent distantes. J'ai compris comment est organisé le cycle de vie complet d'un logiciel industriel – de la spécification à la mise en production.

J'ai aussi appris à faire des compromis : accepter qu'un algorithme optimal en théorie puisse être inadapté en pratique, et qu'une solution simple mais stable est souvent plus précieuse qu'une solution brillante mais fragile. Cette logique industrielle, très différente de celle de la recherche ou de la programmation "en laboratoire", m'a poussé à revoir mes priorités de développeur et à valoriser l'efficacité collective autant que la qualité technique.

Sur le plan professionnel et humain

J'ai aussi beaucoup appris sur le fonctionnement du monde professionnel en France. Grâce à des échanges sincères avec mes collègues, j'ai pu mieux comprendre de nombreux concepts de la vie en entreprise, comme l'organisation du travail dans une équipe distribuée, les rôles respectifs des managers et des développeurs, ou encore les droits et devoirs des salariés – qu'il s'agisse de la durée légale du travail, des congés payés, des RTT, du CSE, des mécanismes de protection sociale et de dialogue social. Ces éléments étaient auparavant assez flous pour un étudiant, et j'ai pu les découvrir dans un climat de bienveillance et de partage.

Je suis particulièrement reconnaissant à mes collègues pour leur accueil chaleureux, leur disponibilité et leur volonté de m'intégrer pleinement à l'équipe. Ils ont partagé avec moi non seulement leur expertise technique, mais aussi leur vécu quotidien en tant qu'ingénieurs, leurs pratiques, leurs habitudes et leur vision du travail. Ces échanges humains ont été aussi formateurs que les compétences techniques acquises.

Enfin, ce stage m'a permis d'évoluer dans ma posture : je me sens aujourd'hui davantage ingénieur que programmeur, conscient que la réussite d'un projet ne repose pas uniquement sur la qualité du code, mais aussi sur la compréhension du contexte, la communication avec les utilisateurs, la coopération avec les collègues, et le respect des exigences du terrain.

Conclusion

Ce stage de fin d'études m'a permis de mettre en pratique mes compétences en algorithmique et en développement logiciel dans un contexte industriel concret. La mission portait sur l'automatisation de l'allocation des câbles dans les connecteurs de coupure, avec des contraintes de compatibilité, d'adjacence et de ségrégation. Pour y répondre, j'ai conçu un algorithme basé sur un parcours en profondeur avec retour sur trace avec un filtrage qui limite l'explosion combinatoire.

Un prototype a d'abord été développé en C++ standard pour valider le comportement et les performances de la solution. Grâce aux choix de conception anticipés, la transition vers CATIA s'est faite de manière fluide. L'algorithme du prototype a été conservé tel quel, avec quelques adaptations liées aux contraintes techniques de la plateforme. La commande finale permet aujourd'hui de proposer automatiquement à l'utilisateur une solution optimale, selon une métrique de regroupement spatial, tout en laissant la possibilité d'ajuster manuellement l'allocation si besoin.

Au-delà de code, j'ai acquis une vision plus claire du cycle de développement d'un logiciel industriel. Ce stage a aussi changé ma manière de penser le développement logiciel. J'ai compris que dans un contexte industriel, la priorité n'est pas l'innovation technique ou la recherche de performance extrême, mais plutôt la stabilité, la maintenabilité et la robustesse des solutions. Cette prise de conscience m'a permis de dépasser une approche parfois trop radicale et théorique, pour adopter une vision plus pragmatique, orientée produit.

Par ailleurs, ce travail a également été l'occasion d'évaluer l'usage de l'intelligence artificielle pour résoudre ce type de problématique. Bien que l'IA représente un outil puissant, son emploi doit être réfléchi et n'est pas adapté à tous les projets. Il est essentiel d'évaluer ses conditions d'application, ses limites et ses performances réelles, notamment en comparaison avec des approches algorithmiques classiques. Dans notre cas, l'absence de données d'apprentissage fiables et la difficulté à garantir un comportement stable en environnement industriel nous ont conduits à privilégier une solution déterministe. Une utilisation mal maîtrisée de l'IA peut non seulement s'avérer inefficace, mais aussi introduire une part d'incertitude susceptible d'engendrer des effets indésirables.

Enfin, sur le plan humain, j'ai mieux compris le fonctionnement d'une entreprise française, ses valeurs, ses règles et ses équilibres sociaux. Grâce à mes collègues, dont les conseils et les échanges m'ont beaucoup enrichi, j'ai pu évoluer dans un cadre à la fois bienveillant et professionnel, propice à l'apprentissage et à l'autonomie.

Je finis ce stage avec des compétences techniques renforcé, une vision plus large du développement logiciel, et une motivation accrue à poursuivre mon parcours professionnel dans le domaine de l'ingénierie logicielle.

Annexes

Annexes A : Référence

Computing Dirichlet tessellations.

Bowyer, Adrian. 1981. 2, Bath : The Computer Journal, 1981, Vol. 24. 105-114.

Computing the n -dimensional Delaunay tessellation with application to Voronoi polytopes.

Watson, David F. 1981. 2, s.l. : The Computer Journal, 1981, Vol. 24.

Smallest enclosing disks (balls and ellipsoids).

WELZL, Emo. 1991. Berlin : Lecture Notes in Computer Science (LNCS, Springer) , 1991, Vol. 555.

Annexes B : Code d'algorithmes

Triangulation Delaunay : Algorithme Boyer - Watson

```
auto delaunay_triangulate(const std::vector<point>& points)
{
    // Construct a super triangle at first.
    auto [p1, p2, p3] = super_triangle(points);
    std::vector<triangle> all_triangles{triangle(p1, p2, p3)};
    // Incremental iteration
    for (const auto& p : points)
    {
        std::set<edge> polygon;
        std::erase_if(
            all_triangles,
            [&](const triangle& current_triangle)
            {
                if (!current_triangle.circum_circle().contains(p))
                    return false;
                else
                {
                    for (const auto& e : current_triangle.get_edges())
                        if (!polygon.erase(e))
                            polygon.insert(e);
                    return true;
                }
            });
        for (const auto& e : polygon)
            all_triangles.emplace_back(p, e);
    }
    // Remove triangles that contain any of the super triangle vertices.
    std::erase_if(
        all_triangles,
        [&](const triangle& current_triangle)
        {
            for (const auto& p : {p1, p2, p3})
                if (current_triangle.containsVertex(p))
                    return true;
            return false;
        });
    return std::move(all_triangles);
}
```

Smallest enclosing circle : Algorithme Welzl

```
circle welzl(std::vector<point>& P, std::vector<point> R, int n)
{
    point p = P[n - 1];
    circle d = welzl(P, R, n - 1);

    if (d.contains(p)) return d;

    R.push_back(p);
    return welzl(P, R, n - 1);
}

double geo_tools::calculate_min_enclosing_circle_radius(std::vector<point> points)
{
    std::mt19937 rng(static_cast<unsigned>(std::time(nullptr)));
    std::shuffle(points.begin(), points.end(), rng);

    circle min_enclosing_circle = welzl(points, {}, points.size());
    return min_enclosing_circle.radius;
}
```

Annexes C : Extension d'infrastructure DS

L'utilisation de la STL a été historiquement interdite chez Dassault Systèmes. Bien que cette restriction ait été levée depuis 2019, la majorité du code existant continue de fonctionner sans recourir à la STL. Cela pose des difficultés lors de la migration et de l'intégration de code algorithmique dans CATIA. Toutefois, il est possible de remplacer les outils de la STL par des boucles et des instructions conditionnelles manuelles. La solution la plus simple est de réécrire ces outils algorithmiques en s'inspirant de l'implémentation de la STL.

`std::erase_if`

C'est une de la fonction la plus utilisée dans l'algorithme de Boyer – Watson pour la triangulation Delaunay. Voici une version minimaliste :

```
template<typename T, typename UnaryPredicate>
void CATOmxArrayEraseIf(CATOmxArray<T>& Arr, UnaryPredicate Pred)
{
    for (int i = Arr.Size(); i > 0; --i)
        if (Pred(Arr[i]))
            Arr.Remove(i);
}
```

Elle donne le même résultat final que `std::erase_if`, mais elle présente une performance médiocre : le fait qu'elle supprime l'élément trouvé dans chaque itération donne une complexité temporelle de $O(n^2)$ dans le pire cas.

Une meilleure solution se trouve dans le code source de STL. L'implémentation de GCC et de MSVC a appliqué la même stratégie : d'abord déplacer les éléments à conserver (donc ceux qui ne satisfont pas le prédicat) vers le début du tableau, ensuite supprimer la partie restante à la fin du tableau en une seule opération. Avec cette méthode on n'a que besoin de réallouer une fois la mémoire et la complexité temporelle devient $O(n)$. L'amélioration des performances est considérable.

```
template<typename T, typename UnaryPredicate>
int CATOmxArrayEraseIf(CATOmxArray<T>& Arr, UnaryPredicate Pred)
{
    int Write = 1;
    int OldSize = Arr.Size();
    for (int Read = 1; Read <= OldSize; ++Read)
        if (!Pred(Arr[Read]))
        {
            if (Write != Read)
                Arr.Set(Write, Arr[Read]);

            ++Write;
        }
    int RemoveCount = OldSize - (Write - 1);
    if (RemoveCount > 0)
        Arr.RemoveAt(Write, RemoveCount);
    return RemoveCount;
}
```

std::vector<T,Allocator>::emplace_back

Cette fonction a pour objectif de simplifier les cas où l'on doit créer un nouvel objet et l'ajouter à un vecteur. Elle permet d'éviter les coûts liés à la création d'un objet temporaire et à sa copie. Pour en profiter, on peut faire une extension pour CATOmXList :

```
template<typename T, typename... Args>
void CATOmXListEmplaceBack(CATOmXList<T>& List, Args&&... args)
{
    CATOmXSR<T> obj(new T(std::forward<Args>(args)...), Steal);
    List.Append(obj);
}
```

Comparaison de nombre à virgule flottant et opérateur « <=> »

[Le célèbre problème de comparaison entre deux nombres à virgule flottant](#) peut être résolu grâce aux macros fournies par Dassault Systèmes.

Il y a cinq opérateurs de comparaison : >, >=, <, <=, =. Le symbole de comparaison trilatérale <=> a été introduit dans C++20, qui est indisponible pour notre projet. Une extension vise à simuler cet opérateur a donc été mis en place :

```
int CATEditorAllocationExtension::DblCompare(double a, double b)
{
    double Diff = std::fabs(a - b);
    double MaxAbs = (std::max)(std::fabs(a), std::fabs(b));

    if (Diff <= CATEpsg || Diff <= MaxAbs * CATEpsg)
        return 0;
    else if (a < b)
        return -1;
    else
        return 1;
};
```

Cette fonction garde le principe de comparaison trilatérale : elle prend deux paramètres A et B, elle retourne -1 si A est inférieur à B, 0 si A est égal à B et 1 si A est supérieur à B. Il n'y a donc plus besoin de définir cinq fonctions ou opérateurs surchargés différents pour la comparaison.